

Kingdom of Saudi Arabia
Ministry of Education
Imam Abdulrahman bin Faisal University
College of Science and Humanities-Jubail
Computer science Department



Software Engineering
Term 1 – 2024/2025

Software Design Specifications

For

HealthyCooking System

Version **1.0**

CS Year 4 - G5 S3

13th of November 2025

This Software Design Specification was prepared and provided as a deliverable for Software Engineering Course, 411, Term 1 – 2025, and it will be used by Group 5 Section 3 team members and their advisor.

This document is based in part on the IEEE Recommended Practice for Software Design Descriptions.

Table of Content

Revision History	2
Table of Tables	5
Table of Figures	6
1. Introduction.....	8
1.1 Purpose.....	8
1.2 Scope	8
1.3 Definitions, Acronyms, and Abbreviations.....	10
1.4 References	11
2. System overview.....	11
3. Design Considerations	12
3.1 Assumptions and Dependencies.....	12
3.2 General Constraints.....	14
4. User Interface Design	15
4.1 Overview of User Interface.....	15
4.2 Interface Design Rules	16
4.3 Screen Images	17
4.4 Screen Objects and Actions	30
4.5 Other Interfaces.....	32
5. System Architecture.....	37
5.1 Architectural Design Approach	37
5.2 Architectural Design	38
38	
5.3 Subsystem Architecture	39
6. Data Design	45
6.1 Data Description	45
6.2 Data Dictionary	45
6.3 Database Description	46
7. Component Design	49
7.1 User Authentication Component	49
7.2 User Profile Component	52
7.3 Recipe Management Components	54
7.4 Meal Planner Component	58
7.5 Calorie Tracker Component	60
7.6 Notification Component	62
8. Detailed System Design.....	64
8.1 Classification.....	64
8.2 Definition	65
8.3 Responsibilities	66
8.4 Constraints	68
8.5 Composition.....	69
8.6 Uses/Interactions.....	70
8.7 Resources	80
8.8 Processing	81
8.9 Interface/Exports.....	81
8.10 Detailed Subsystem Design	83

9.	Other Design Features	84
10.	Requirements Traceability Matrix	84

Table of Tables

Table 1: Software Design Specification Document Acronyms	10
Table 2: End user characteristics	13
Table 3: Login interface objects	30
Table 4: Sign-up interface objects	30
Table 5: Homepage interface objects	30
Table 6: Profile and settings interface objects	30
Table 7: Add recipe interface objects	30
Table 8: Forget password interface objects	31
Table 9: Favorites interface objects	31
Table 10: Edit recipe interface objects	31
Table 11: Plan weekly meals interface objects	31
Table 12: Personalized meal recommendations interface objects	31
Table 13: Data description of HealthyCooking system	45
Table 14: Local data of user authentication Component	51
Table 15: local data of the user profile component	53
Table 16: Local data of recipe management component	57
Table 17: Local data of meal planner component	59
Table 18: Local data of calorie tracker component	61
Table 19: Local data of notification component	63
Table 20: Components classifications	64
Table 21: Components definitions	65
Table 22: Components resources	80
Table 23: Most important system functions	82
Table 24: Subsystems design reference	83
Table 25: Requirements Traceability Matrix	85

Table of Figures

Figure 1: sign-up interface.....	17
Figure 2: log-in interface	18
Figure 3: Main homepage interface.....	19
Figure 4: Main homepage interface.....	19
Figure 5: Add recipe interface	20
Figure 6: Add recipe interface	20
Figure 7: Delete recipe interface that shows confirming recipe deletion from favorite section.....	21
Figure 9: Delete recipe interface.....	21
Figure 8: deletion interface that shows confirmation message stating that deletion was successful	21
Figure 10: Plan weekly meals interface.....	22
Figure 11: Plan weekly meals interface.....	22
Figure 12: Plan weekly meals interface.....	22
Figure 13: Plan weekly meals interface.....	22
Figure 14: Edit recipe interface	23
Figure 15: Edit recipe interface	23
Figure 16: Track calories interface	24
Figure 17: Meal reminders interface.....	25
Figure 18: Personalized Meal Recommendations Interface	26
Figure 19: Profile management interface	27
Figure 20: Rest forgotten password interface.....	28
Figure 21: Favorite recipes interface	29
Figure 22: Additional sign-up interface.....	32
Figure 23: Additional sign-up interface.....	32
Figure 24: Additional forget password interface	33
Figure 25: Additional log-in interface	34
Figure 26: Additional profile management interface.....	35
Figure 27: Additional profile management interface.....	35
Figure 28: Additional delete recipe interface	36
Figure 29: The HealthyCooking system architecture design approach.....	37
Figure 30: The HealthyCooking system design architecture.....	38
Figure 31: Context DFD of HealthyCooking system	39
Figure 32: User authentication subsystem functionalities	40
Figure 33: User profile subsystem functionalities	41
Figure 34: Recipe management subsystem functionalities.....	42
Figure 35: Meal planner subsystem functionalities	43
Figure 36: Calorie tracker subsystem functionalities	44
Figure 37: HealthyCooking ER Diagram	47
Figure 38: HealthyCooking Relational Schema	48
Figure 39: pseudocode of the sign-up function	49
Figure 40: pseudocode of the log-in function.....	50
Figure 41: pseudocode of the rest password function	50
Figure 42: pseudocode of the log-out function.....	51
Figure 43: pseudocode of the update profile function.....	52

Figure 44: pseudocode of the view profile function	52
Figure 45: pseudocode of the add recipe function.....	54
Figure 46: pseudocode of the update recipe function	54
Figure 47: pseudocode of the delete recipe function	55
Figure 48: pseudocode of the view recipe function.....	55
Figure 49: pseudocode of the search recipe function	56
Figure 50: pseudocode of the generate weekly plan function	58
Figure 51: pseudocode of the view weekly plan function	58
Figure 52: pseudocode of the add meal function.....	60
Figure 53: pseudocode of the calculate daily calorie's function	60
Figure 54: pseudocode of the view daily summary function.....	61
Figure 55: pseudocode of the send meal reminder function.....	62
Figure 56: pseudocode of the send calorie alert function	62
Figure 57: User Login Sequence Diagram	70
Figure 58: Update profile and preferences sequence diagram.....	71
Figure 59: Generate Weekly Meal Plan Sequence Diagram	72
Figure 60: Track Calories and Send Alert Sequence Diagram.....	73
Figure 61: Browse and Search Recipes Sequence Diagram	74
Figure 62: View Recipe Details Sequence Diagram	75
Figure 63: Add Recipe to Personal Collection Sequence Diagram.....	76
Figure 64: Save Recipe to Favorites Sequence Diagram.....	77
Figure 65: Remove Recipe from Favorites Sequence Diagram	78
Figure 66: Generate AI Recipe Suggestions Sequence Diagram	79

1. Introduction

This SDS document shall present the complete software design for the HealthyCooking System 1.0 version. It develops the functional and non-functional requirements specified in the SRS document [1] into an overall design structure that describes how the system will be implemented and how its various components will interact.

The HealthyCooking System is designed as a mobile application aimed at maintaining good eating behavior and supporting users in managing nutrition and meal planning based on reliable dietary standards from the World Health Organization (WHO) [2].

The following document outlines the software architecture, component interaction, data design, and interface layout that, taken all together, ensure that the system is scalable, secure, and user-friendly.

This SDS follows the guidelines set forth in the IEEE Standard for Software Design Description, IEEE Std 1016-2009 [3], and it is meant to be a blueprint that the developers, testers, and project evaluators will follow to implement the system as specified in the SRS with accuracy.

The subsections below include the Purpose, Scope, Definitions, Acronyms, Abbreviations, and References related to this document.

1.1 Purpose

The purpose of this SDS is to describe how the HealthyCooking system will be built to satisfy the requirements outlined in the SRS [1]. This document defines the software architecture, internal and external interfaces, database structure, and user interface design. It is the technical blueprint of how certain user needs, like the exploration of recipes, meal planning, and tracking of nutrition, will be realized in an implemented system.

Also, the SDS makes sure that all team members view the system structure from the same perspective and guides developers and testers to implement the design consistently with predefined requirements [3].

1.2 Scope

This SDS describes the overall design strategy of the HealthyCooking application. It consists of the architectural layout, module description, user interface design, and database design necessary to support the primary objectives of the system, that is, a balanced diet and personalized meal planner [2].

The system incorporates features such as:

- Management and Integration of Recipe Database.
- Personalized nutrition tracking.
- Clever meal planning based on users' preferences.
- Manage and store data securely.

This document serves as the design base for further development and focuses on maintainability, extendibility, and adaptability in view of future improvement of the system.

Intended Audience

This Software Design Specification document is targeted at a wide variety of people and technical roles that might be involved in planning, developing, evaluating, and maintaining the HealthyCooking system in the long run. It will provide a detailed reference for software designers and system architects who need an in-depth understanding of the structural and functional organization of the system if their task is to produce a consistent and well-aligned implementation.

This document further addresses the developers who will translate the described design elements, modules, interfaces, and data structures into executable code. The SDS will act as a guideline for testers and quality assurance engineers to design test cases, verify system behavior, and ensure that the system implemented achieves full compliance with the SRS specifications.

This SDS will be used by project supervisors, course instructors, and academic evaluators as a reference for the purpose of reviewing correctness, completeness, and conformance to professional and academic standards such as the IEEE 1016. It may also be referred to by stakeholders or decision-makers needing to comprehend the design rationale of the system and/or the architectural decisions and constraints that will affect future enhancements.

Finally, this SDS may become supporting documentation for any teams that will either maintain or extend the system, providing the necessary technical background and structural insight to foster continuity, consistency, and scalability in its evolution.

1.3 Definitions, Acronyms, and Abbreviations

The HealthyCooking Software Design Specification (SDS) document includes several acronyms and abbreviations, which are defined in this document. This ensures that all project participants have a consistent understanding of the terminology used throughout the document.

Acronym / Term	Definition
AI	Artificial Intelligence: Algorithms that learn and generate intelligent recommendations.
API	Application Programming Interface: Channel that enables communication between software components.
SRS	Software Requirements Specification: a document that describes all functional and non-functional requirements of the system.
SDS	Software Design Specification: A document that describes the architecture and design of a software system.
BMI	A person's height and weight are used to calculate their body mass index, a number that is used to evaluate their health and body fat.
IEEE	Institute of Electrical and Electronics Engineers: Developer of key software engineering standards.
UI/UX	User Interface and User Experience: Design principles ensuring clarity and usability.
DB	Database: Central storage of all recipe data, user information and nutrition data.
DBMS	Database Management System :A software system used to store, organize, manage, and retrieve data efficiently for the HealthyCooking application.
ERD	Entity Relationship Diagram: A visual representation of database tables and their relationships in the HealthyCooking system.
UAT	User Acceptance Testing: A testing process that ensures the system meets user needs and requirements before deployment.
CRUD	Create, Read, Update, Delete: The four basic operations performed on data in the HealthyCooking system.
WHO	World Health Organization: A global health authority whose dietary and nutritional guidelines are used in this system.
Pseudocode	A natural English way to describe how an algorithm works without using real code.
Local Variables	A variable that exists only inside a specific function or block of code, then disappear after the function finishes.
Authentication	The process of checks user's identity, as confirming the Email and password.
Authorized Users	Users who are allowed to access features because they have the permissions.
Encryption	A method of turning data into a secret, unreadable form so only authorized people can read it.

Table 1: Software Design Specification Document Acronyms

1.4 References

- [1] HealthyCooking Software Requirements Specification (SRS) Document, 2025.
- [2] World Health Organization (WHO), Healthy Diet: Key Facts, Geneva, Switzerland, 2020. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/healthy-diet>
- [3] IEEE Computer Society, IEEE Standard for Information Technology – Software Design Descriptions (IEEE Std 1016-2009), New York, USA: IEEE, 2009.
- [4] Bhasker Reddy Bhimanpati, P. K. Gopalakrishna Pandian, and P. Goel, “UI/UX Design Principles for Mobile Health Applications,” *Int. J. Res. Publ. Seminars*, vol. 15, no. 3, pp. 216–231, Aug. 2024. doi: 10.36676/jrps.v15.i3.1485.

2. System overview

It is a mobile application software meant for enabling and supporting healthy eating through nutritional management and personalized meal planning. In order to provide such an effective solution to end-users interested in changing their eating habits, the system combines a user-friendly interface with smart algorithms and a structured recipe database. System Architecture The system proposed would have the following high-level architecture:

1. Presentation / User Interface Layer:

This interface navigates between recipes, weekly planning, and daily nutrition tracking to make it easy for the user to interact with the system.

2. Application Logic Layer:

Processes user input for nutritional analysis and offers recommendations based on users' preferences and health goals.

3. Database Layer:

This layer ensured the secure storage of recipes, user profile data, and nutritional information with scale and accuracy of data.

The system is designed for users to plan meals, discover recipes, and track nutrition. HealthyCooking would hold intelligent meal-planning algorithms for suggestions aligned with food preferences, calorie intake, and dietary guidelines provided by the World Health Organization [2]. This architecture is highly scalable and provides flexibility to expand into advanced analytics or other dietary modules. HealthyCooking combines functionality with simplicity and reliability, thus offering an efficient and accessible way of managing one's daily nutrition effectively with guaranteed data security and performance as outlined in SDS [3].

3. Design Considerations

The following section highlights various design issues, their dependencies, and architectural constraints that need to be resolved before the implementation of the HealthyCooking System; it ensures that all design decisions are consistent with the system goals related to the provision of nutrition analysis, meal planning, and an easy user experience.

Every consideration of the design was done using the best practices of software engineering and following the IEEE Software Design Description [3]. These decisions have been made to balance functionality, performance, and scalability while ensuring system integrity and usability.

3.1 Assumptions and Dependencies

The following assumptions and dependencies were considered during the system design. They define the expected operational environment and external factors that will influence system behavior.

Platform and Operating System

The HealthyCooking system is designed as a mobile application that can be downloaded and accessed through iOS and Android operating system mobiles. It supposes that users will have a stable internet connection and can connect to the system from anywhere and on any device. This flexibility ensures that the accessibility to health-conscious persons and nutrition experts is high.

Hardware Requirements

All end users should have mobile devices that run the operating system smoothly. In terms of hardware resources, this system does not require many: even a standard processor, 4 GB RAM, and a stable network will suffice for its full functionality. As the system does real-time nutritional calculations and data retrieval, performance is essentially dependent on internet speed and stability.

Database Dependency

This system would be highly dependent on a Database Management System for the storage and retrieval of information regarding users, recipes, and nutrition records. The backbone of HealthyCooking is the database, in which meal data, ingredients, and user preferences are stored efficiently. Any unavailability or inconsistency of data in the DBMS will directly impact the recommendation of meals or accuracy in user tracking.

Third-Party Dependencies

This work on application intelligence also includes the integration of external APIs: for example, nutrition fact APIs, calorie calculation APIs, and ingredient substitution APIs. These APIs are assumed by the system to always be up, compatible, and reliable for continuous operation and accurate results.

User Characteristics

This assumes that system users possess the basic digital literacy needed to navigate through recipes, input data, and interpret visual nutritional feedback. The design also considers accessibility for both

casual users and professional diet planners, ensuring that the interface remains intuitive for all levels of users.

End-User Characteristics

User Type	Age	Gender	Education Level	Computer Skills	Region
User	Above 17	Male/Female	High School or Above	Good	All around the world

Table 2: End user characteristics

Upcoming System Improvements

Future updates may include diet tracking statistics and transferring user progress to fitness applications or integrating recipes from outside nutrition sources. Thanks to the system architecture's modular scalability. This guarantees that the design stays adaptable without requiring significant re-engineering.

3.2 General Constraints

HealthyCooking is subject to a number of environmental and technical limitations that affect implementation tactics and design decisions.

Environment of Hardware and Software Constraints

Without the need for extra installations, the program is made to increase data availability and reduce hardware reliance on client systems. It makes use of cloud-based storage.

Security and Privacy Constraints

The system adheres to stringent data protection guidelines because it stores important food and health information. Every user's data is encrypted both during transmission and storage. In order to protect privacy and stop unwanted data exposure, access is limited to authorized users.

Performance Requirements

For operations like meal planning, nutrition tracking, and recipe search, the system must maintain quick response times, ideally less than two seconds each request. Performance testing will guarantee that the system is stable and responsive even when there are several concurrent sessions.

Data Integrity and Storage

The relational database schema (ERD) used by HealthyCooking preserves referential integrity between user, recipe, and nutrition tables. To avoid duplication or inconsistency, each data update is validated before being committed.

Scalability

Future increases in the number of users and saved recipes are supported by the design. A Meal Recommendation Engine or Diet Progress Tracker, for example, can be added without compromising fundamental functionality because to the modular architecture.

Usability

Following UI/UX design principles, the interface emphasizes readability, simplicity, and low cognitive burden. Clean visual layouts and simple navigation make it simple for users to access key features like adding meals, reading recipes, and tracking nutrition.

Interoperability

HealthyCooking uses standardized APIs to interface with other systems. This makes it possible to interchange data across platforms.

Validation and Verification

User acceptability, integration, and unit testing will all be applied to each module. While validation guarantees that the system operates correctly under real-world circumstances, verification guarantees that the system design complies with the requirements specified in the SRS [1].

4. User Interface Design

4.1 Overview of User Interface

The purpose of the HealthyCooking system is to give users an interface that will easily and intuitively help them to browse recipes, manage their meals for the week, and help them to track their intake of nutrients. All types of users should navigate through the interface without facing any complications. According to users, the system offers the following:

Recipe Exploration:

It shows recipes categorized according to meals with their ingredients, nutritional value.

Meal planning:

It enables users to easily choose recipes, then set them in a daily schedule to plan meals for a week. Through this interface, one receives a calendar of meals and summaries of nutrition.

Nutrition tracking:

This feature will allow the user to track their progress daily, weekly, and more, through an interactive dashboard, which will display calorie counts, distribution of macronutrients, and health indicators.

User profile management:

Users can change personal information like preferences, diet, calorie goals, and health restrictions that affect the recommendations.

4.2 Interface Design Rules

The interface design of HealthyCooking follows the same UI/UX principles [4] throughout the application so that it is user-friendly, clear, and easy to access.

Consistency:

Consistent color palettes, typography, layout spacing, and component styling have been used in all the pages to maintain a unified system identity.

Simplicity:

The interface minimizes visual clutter. To ensure ease of navigation and discovery of features, buttons, forms, and navigation elements have been logically placed.

Accessibility:

The design choices take into consideration users of varying levels of technical expertise, readable font sizes, and suitable contrast between text color and background

colors:

Intuitive icons and labels, and Mobile-friendly layouts.

Feedback:

Users receive clear feedback, such as success and error messages.

Navigation Rules:

The system has a navigation that helps users go through the system. The user can access recipes, meal plans, and dashboards from the main menu. The user can always go back home.

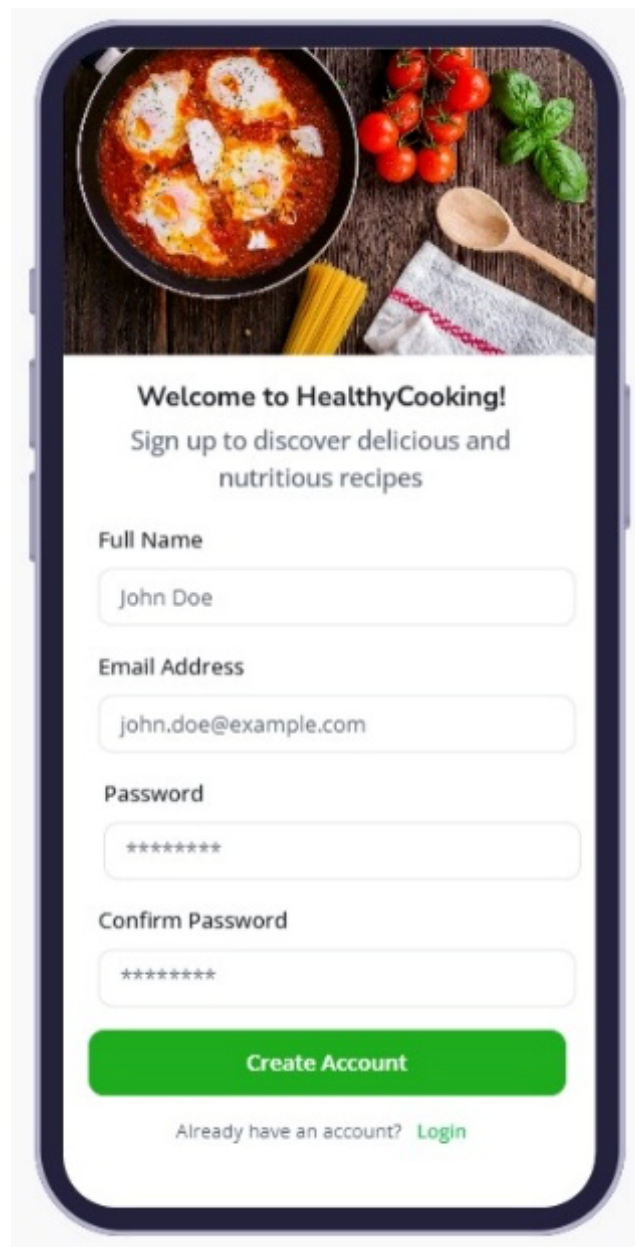
Form and Input Design:

Forms follow standard rules, such as required fields being clearly marked, validation messages being shown immediately, suggesting some inputs, or giving examples when needed, security, and data Handling.

4.3 Screen Images

Sign-up Interface:

Here is the registration interface is presented to us as a welcome for the users of our program. It shows the user that they must enter their name and also their email address along with a unique password to ensure unauthorized access is prevented. There is also a field to confirm the entered password, and the user presses the button to complete creating their account.



Welcome to HealthyCooking!
Sign up to discover delicious and nutritious recipes

Full Name

Email Address

Password

Confirm Password

Create Account

Already have an account? [Login](#)

Figure 1: sign-up interface

Log-in Interface:

This interface allows registered users to access their accounts by entering their email address and password. It provides a clean layout with two input fields and a main button to submit the login request. The user can also choose to reset their password if needed through the “Forgot Password” option displayed beneath the fields.

The image shows a mobile application login screen. At the top, there's a header image of a dish with the text "Welcome Back!". Below this, there are two input fields: "Email Address" containing "john.doe@example.com" and "Password" containing masked characters "*****". A note below the password field states "Minimum 8 characters including letters, numbers, and symbols". A green "Log In" button is positioned below the password field. To the right of the button is a green link "Forgot Password?". At the bottom of the screen, there is a link "Don't have an account? Sign up".

Figure 2: log-in interface

Main Home Page Interface:

This main interface features a recipe search box, a bar that displays content and options, as well as AI suggestions for the user. The suggestions include user requests and show what is suitable for them based on allergens, user preferences, and their previous recipes. There is also a menu with all available recipes, providing the time required, calories, and a brief description.

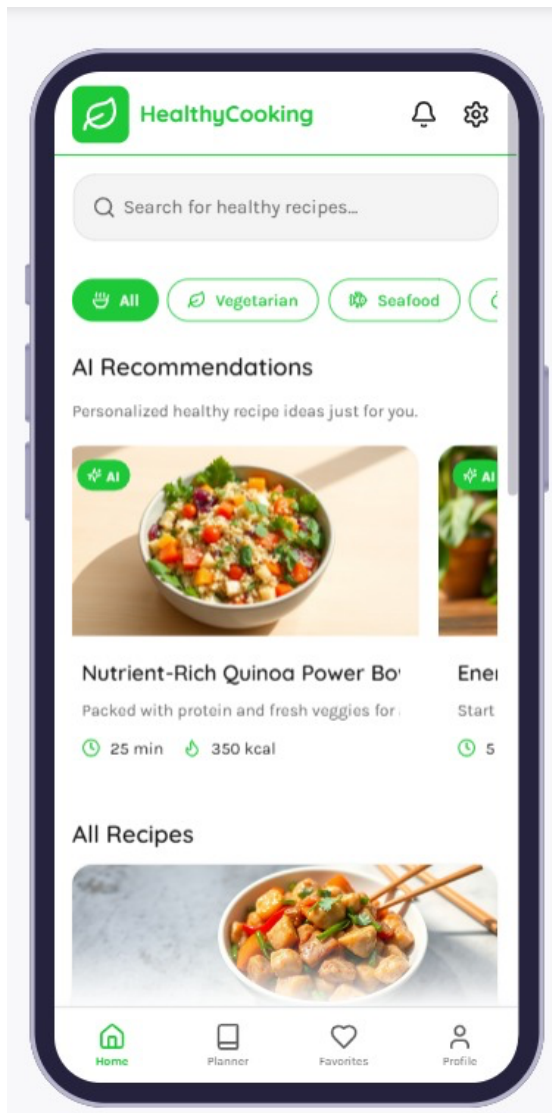


Figure 4: Main homepage interface

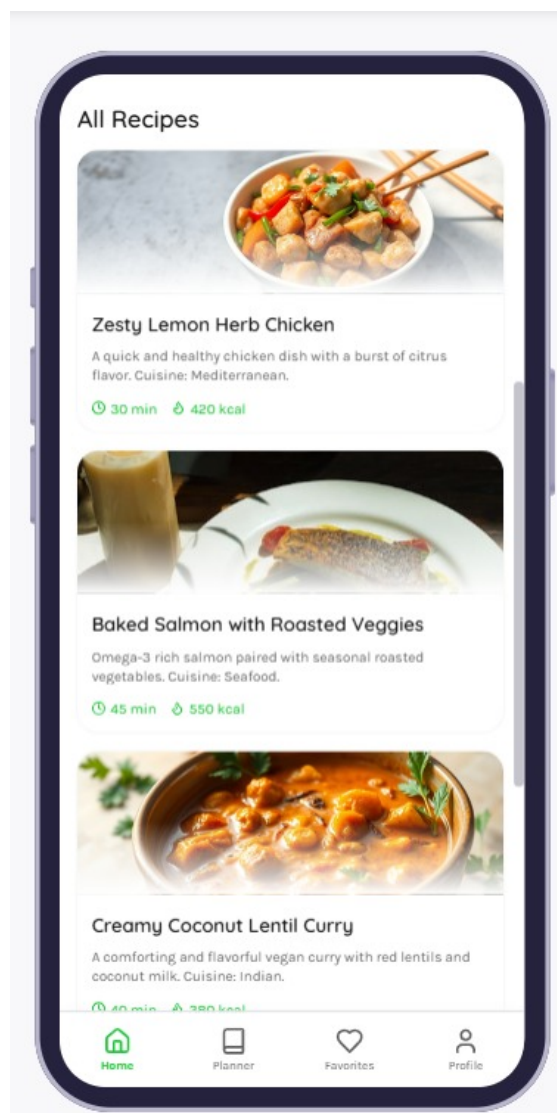


Figure 3: Main homepage interface

Add Recipe Interface:

The add interface displays when the user selects the recipe that suits them; it will show them the description, steps, instructions, and also the nutritional information.

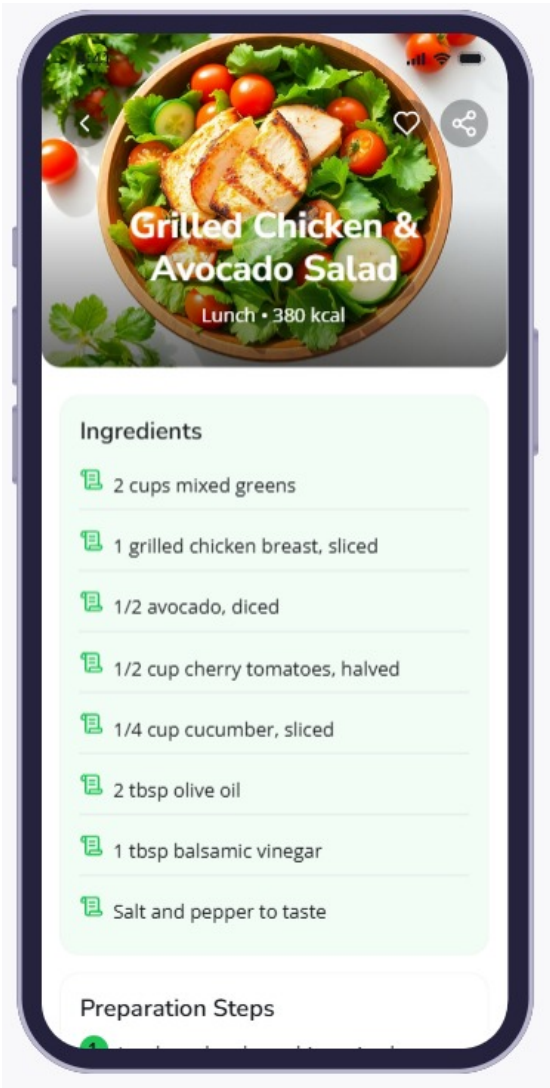


Figure 6: Add recipe interface

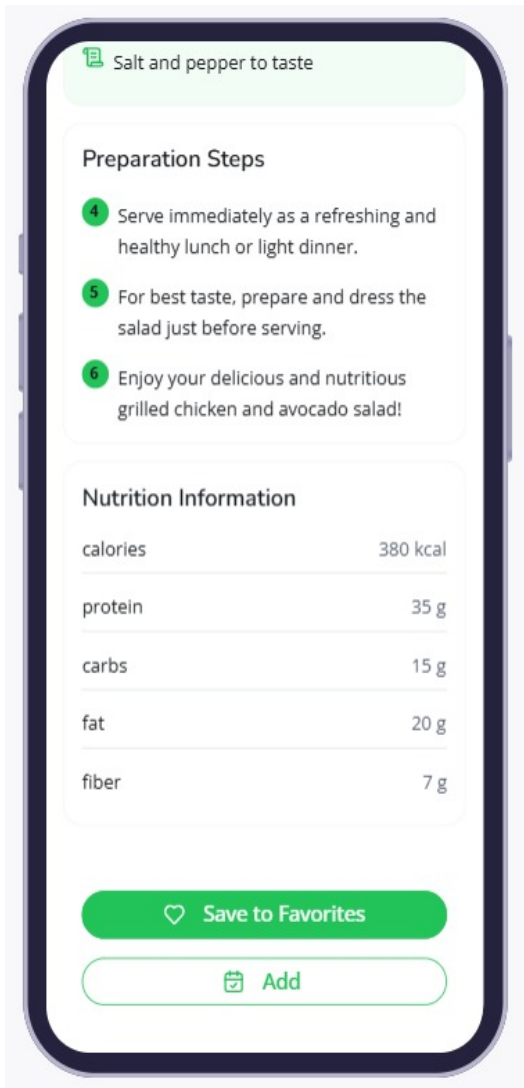


Figure 5: Add recipe interface

Delete Recipe Interface:

Remove button for delete recipes from favorites section. Then, a page appears confirming the recipe deletion, displaying the dish name and a button to confirm or cancel the deletion. Finally, a confirmation page appears stating that the deletion was successful.

Open button for page opens recipe It will show whether the recipe was successfully deleted after pressing the delete button, or if the open button was pressed, it will display a deletion failure if the recipe deletion was not confirmed.

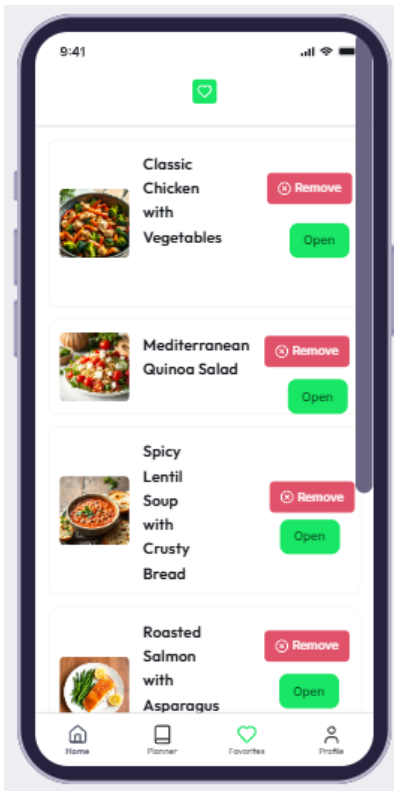


Figure 9: Delete recipe interface

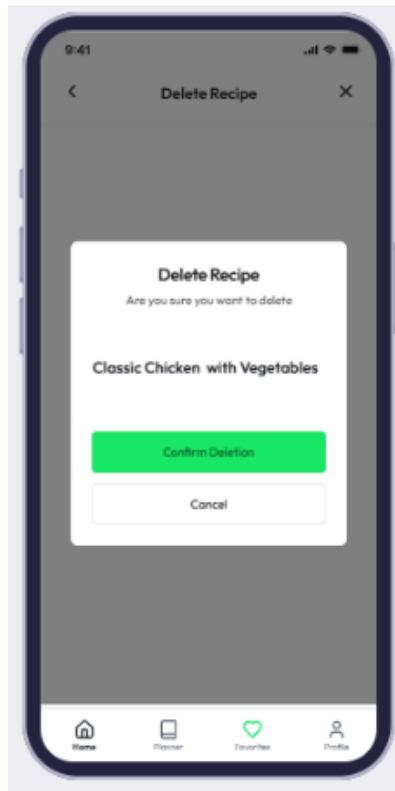


Figure 8: deletion interface that shows confirmation message stating that deletion was successful

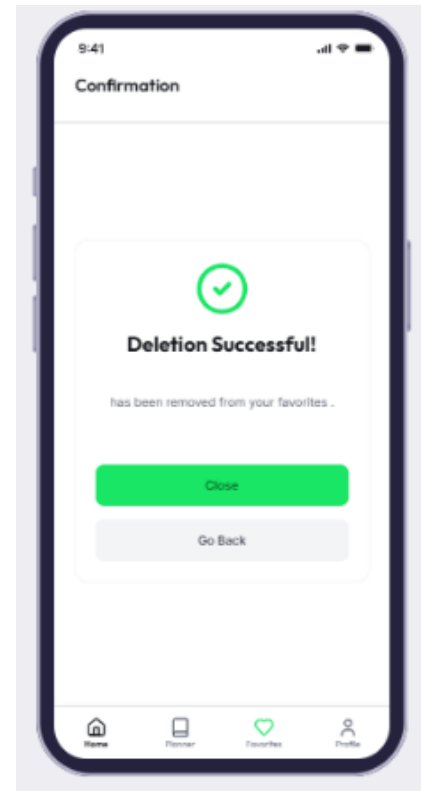


Figure 7: Delete recipe interface that shows confirming recipe deletion from favorite section

Plan Weekly Meals Interface:

This interface allows users to plan their weekly meals by selecting a day, choosing a recipe, and assigning it to a specific meal (breakfast, lunch, or dinner). The design guides the user step-by-step to build their weekly schedule, and once the planning is saved, the system confirms successful completion.

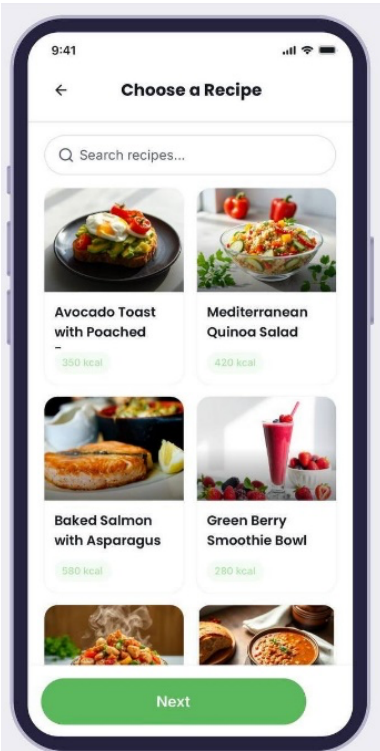


Figure 12: Plan weekly meals interface

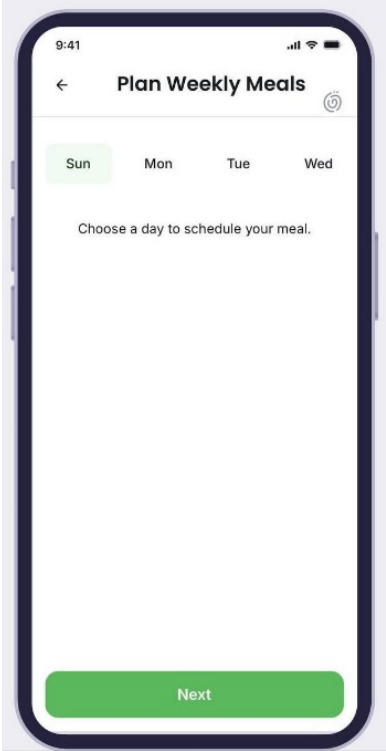


Figure 10: Plan weekly meals interface

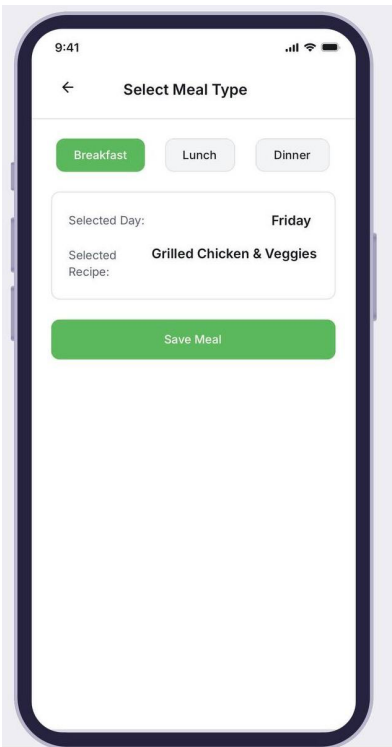


Figure 11: Plan weekly meals interface

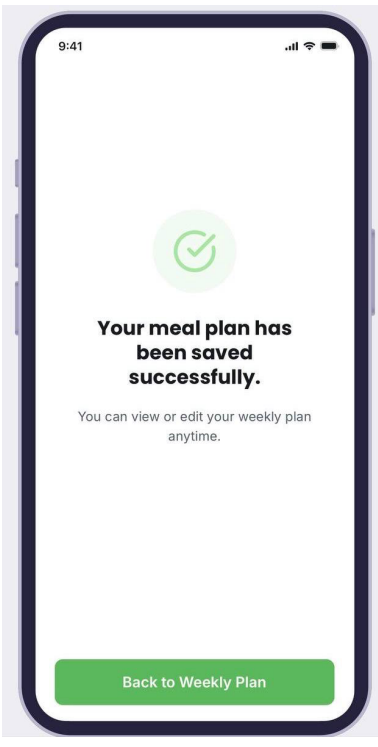


Figure 13: Plan weekly meals interface

Edit Recipe Interface:

This interface enables users to modify an existing recipe. It displays the recipe image, title, list of ingredients, and cooking steps, all in editable fields. The user can update the content, replace the recipe image, save the new changes, or delete the recipe entirely if they no longer wish to keep it.

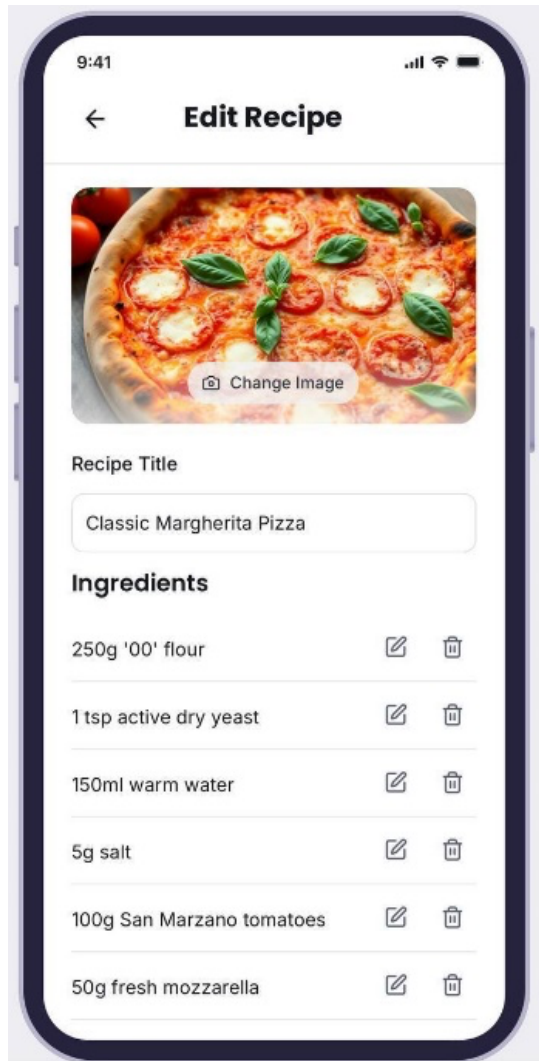


Figure 14: Edit recipe interface

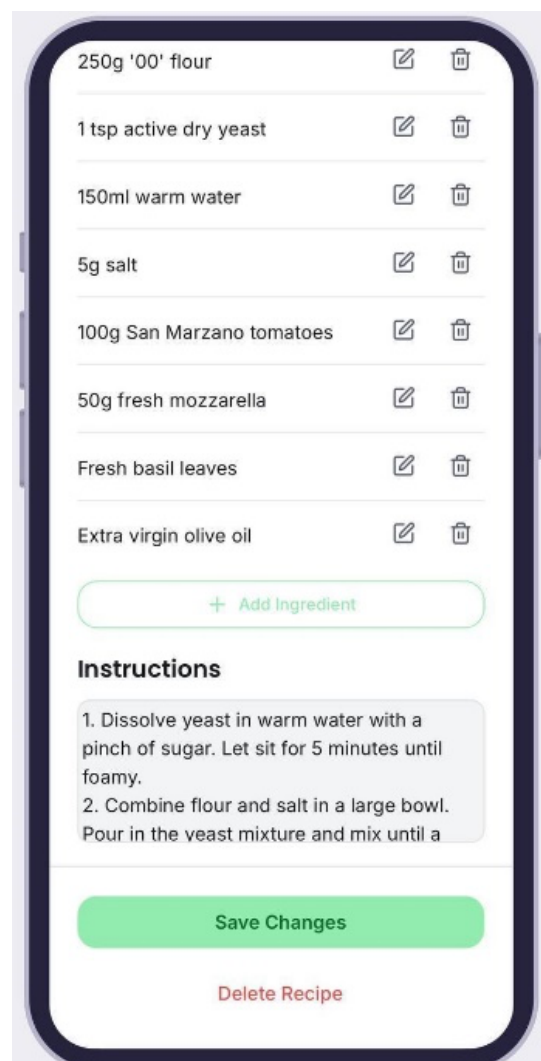


Figure 15: Edit recipe interface

Track Calories Interface:

Track calories for each meal breakfast, lunch, dinner, and snack. This feature allows users to track their calories throughout adding the meals they had in the system.

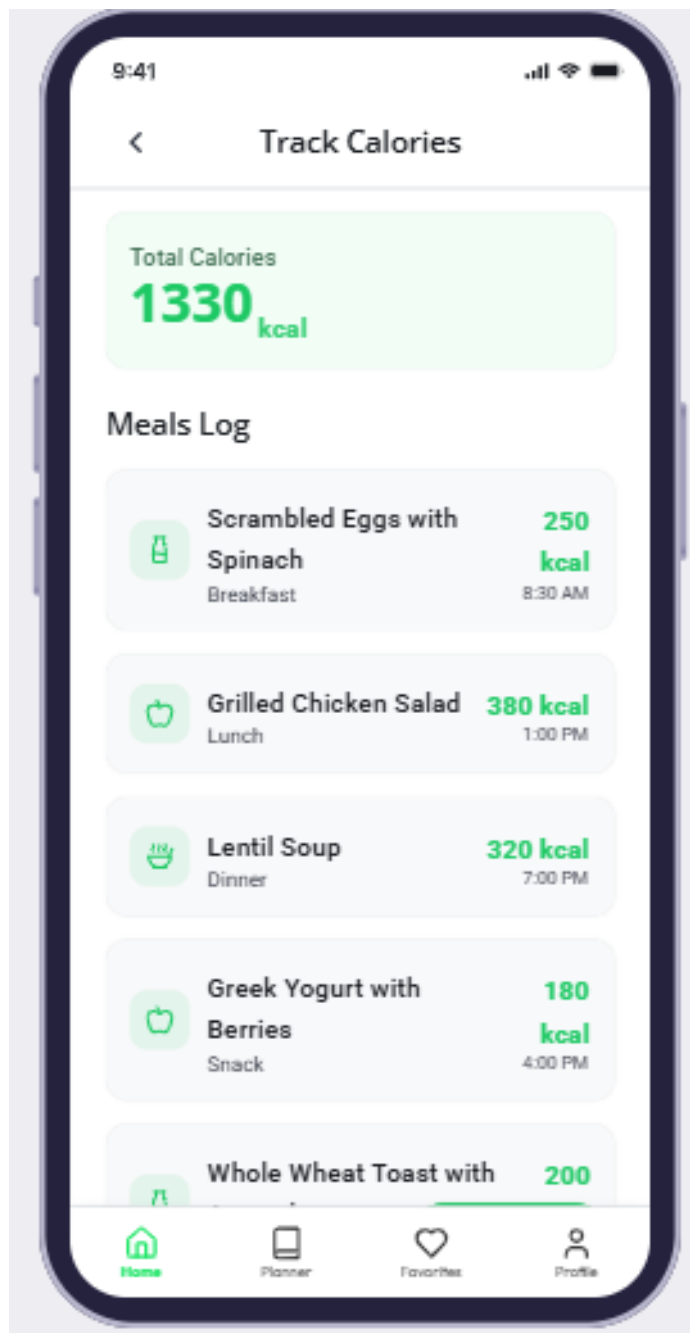


Figure 16: Track calories interface

Meal Reminders Interface:

The HealthyCooking application provides meal reminder interface that allows users to view, edit, delete, and add meal notifications. Each meal breakfast, lunch, dinner, snack is displayed with its scheduled time, and users can easily manage their daily meal schedule through quick access action icons and an add button for creating new reminders.

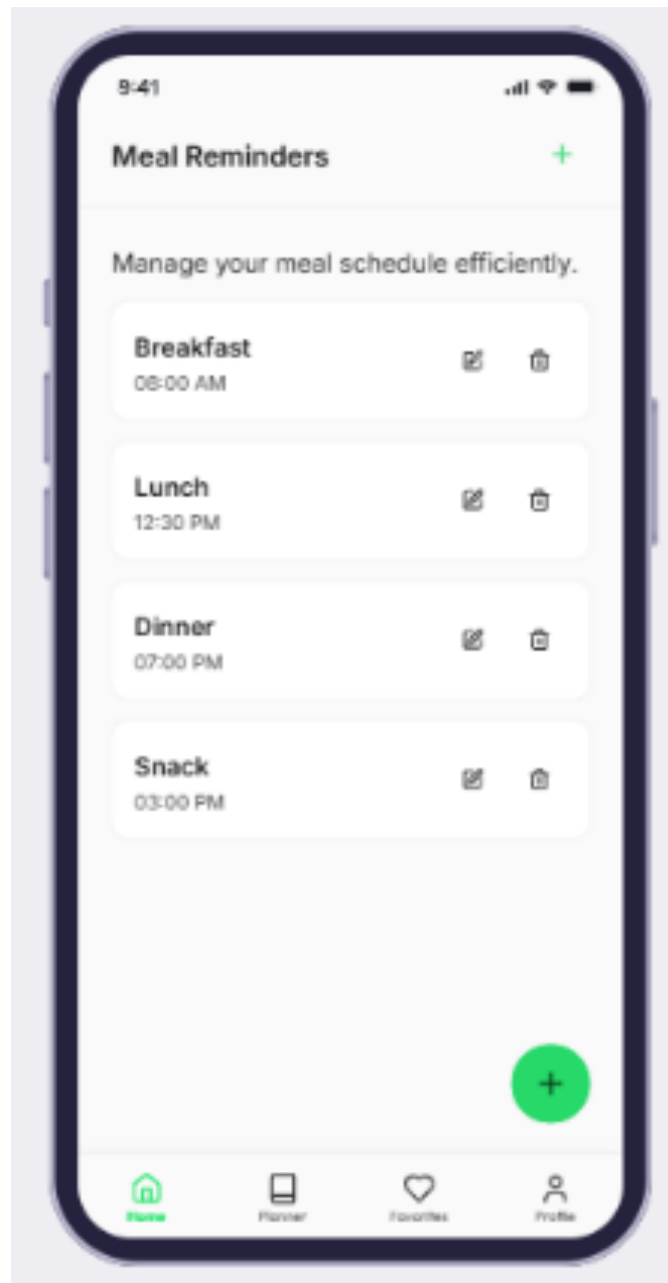


Figure 17: Meal reminders interface

Personalized Meal Recommendations Interface:

This interface displays meal recommendations generated specifically for the user based on their preferences, allergies, nutritional goals, and previously saved or liked recipes. Each recommended meal appears in a card format, showing an image, title, calorie information, and quick action icons such as Like and Save.

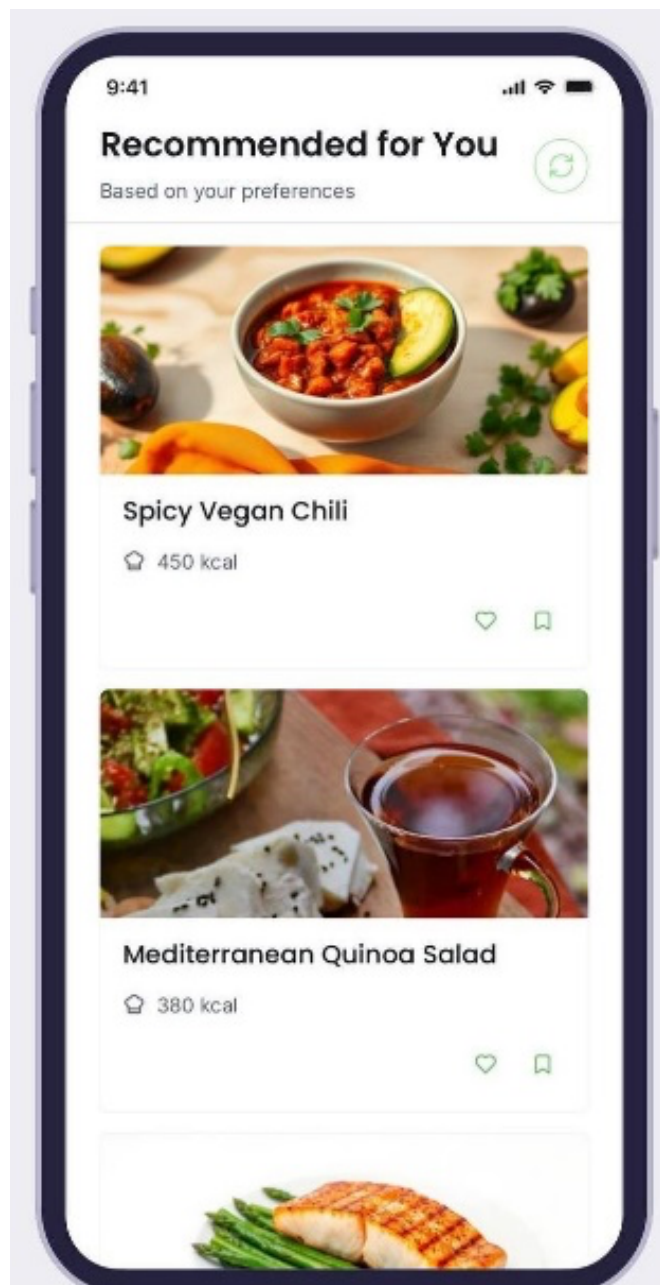


Figure 18: Personalized Meal Recommendations Interface

Profile Management Interface:

In the profile page, we display the username, allergy triggers, and ingredients that the user prefers to avoid, along with a button to save the information when it is updated.

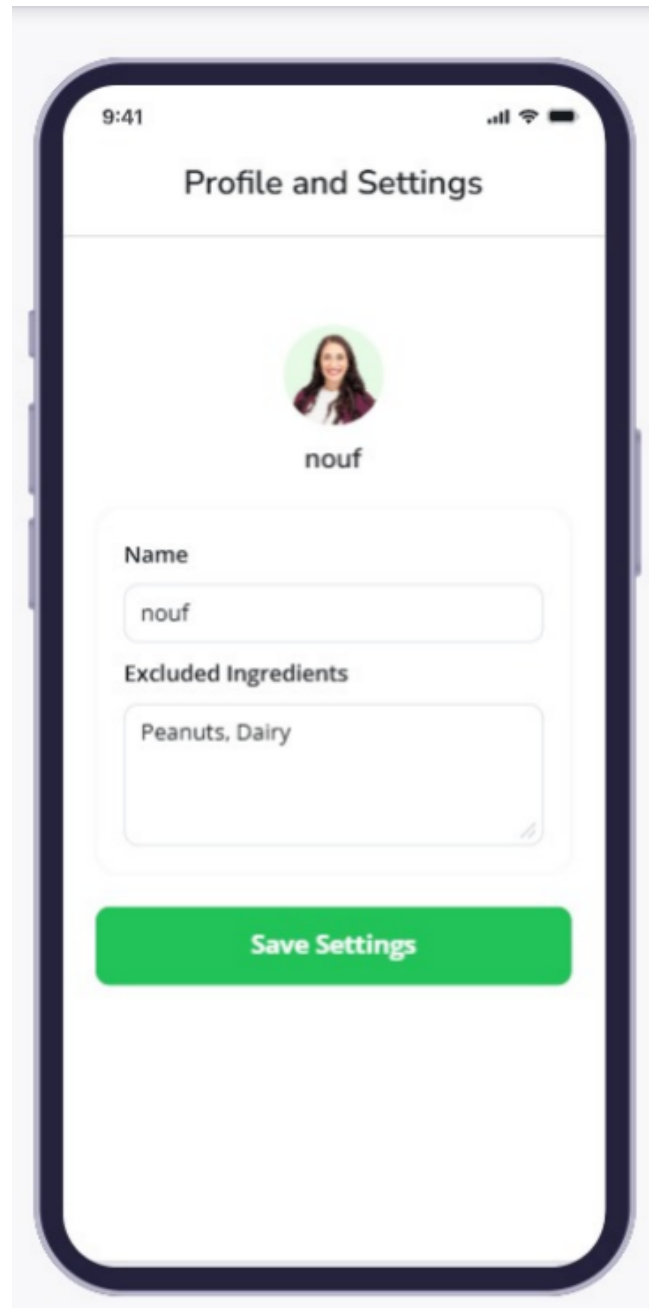


Figure 19: Profile management interface

Forgot Password Interface:

When the user wants to reset their password, they will need to enter their email to verify and update the new password.

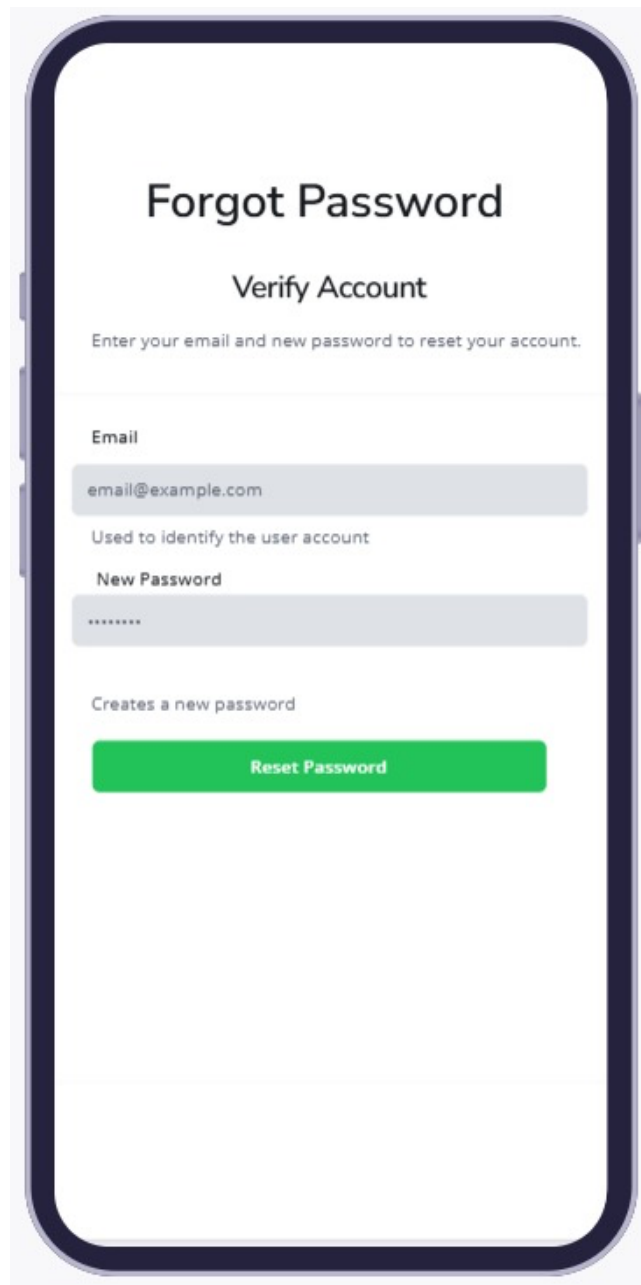
A mobile app interface for a 'Forgot Password' screen. The screen has a dark blue rounded rectangle border. At the top, the title 'Forgot Password' is centered in a large, bold, black font. Below it, the subtitle 'Verify Account' is centered in a smaller, bold, black font. Underneath the subtitle, a line of text reads 'Enter your email and new password to reset your account.' in a small, gray font. The form consists of two main input sections. The first section is labeled 'Email' in a small, bold, gray font. It contains a light gray rounded rectangle input field with the text 'email@example.com' in a small, gray font. Below the field, a line of text reads 'Used to identify the user account' in a small, gray font. The second section is labeled 'New Password' in a small, bold, gray font. It contains a light gray rounded rectangle input field with seven dots '*****' in a small, gray font. Below the field, a line of text reads 'Creates a new password' in a small, gray font. At the bottom of the form, there is a solid green rounded rectangle button with the text 'Reset Password' in a white, bold, sans-serif font.

Figure 20: Rest forgotten password interface

Favorites Interface:

Here Display recipes saved by the user for quick access to recipes.

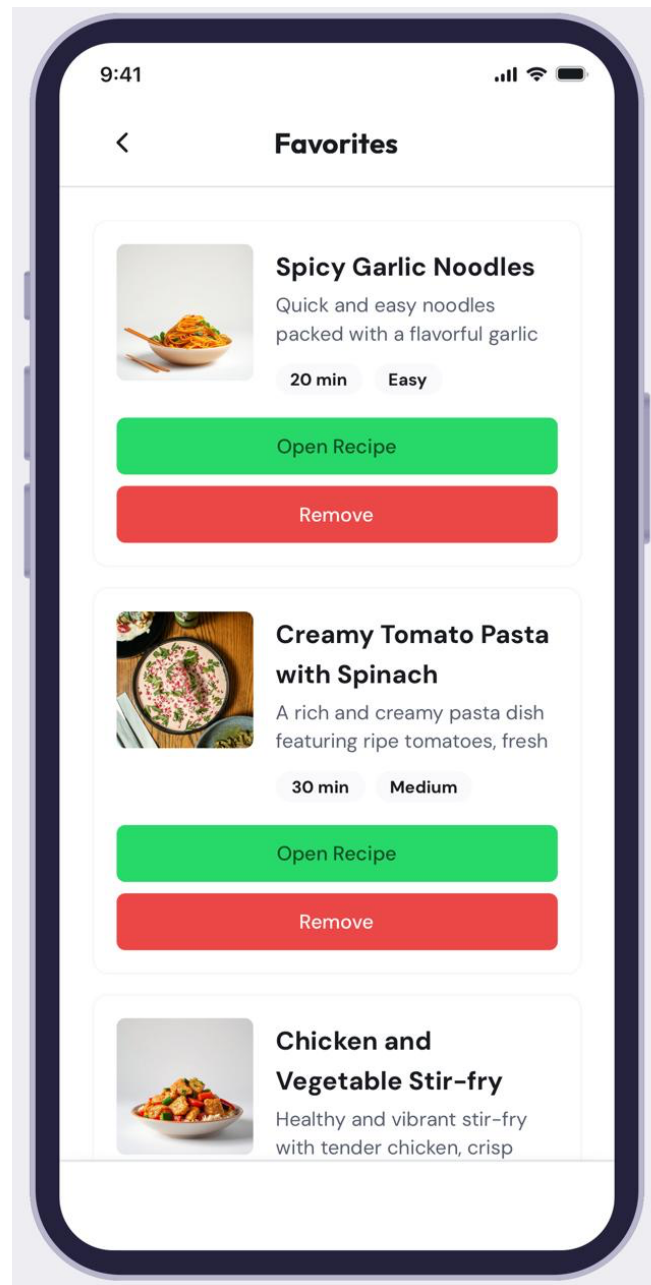


Figure 21: Favorite recipes interface

4.4 Screen Objects and Actions

Login Interface			
Input Field	Description	Type	Requirement
Email	Registered user email	Text	Required – must be unique
Password	User password	Encrypted Text	Required – minimum 8 characters including letters, numbers, and symbols

Table 3: Login interface objects

Sign-up Interface	
Field	Data Type
Full Name	Text
Email	Text
Password	Encrypted Text
Confirm Password	Encrypted Text

Table 4: Sign-up interface objects

Home Interface	
Component	Function
Search box	The search box allows searching by ingredient, recipe name, or cuisine type.
Filter	Our program filters recipes by cuisine type, specific dish type, and preparation time in minutes or hours.
Recipe	Displays recipe image, title, and calorie info
Navigation Bar	Provides links to Home, Planner, Favorites, and Profile

Table 5: Homepage interface objects

Profile and Settings Interface			
Items	Function	Format	Type
Name	Displays or edits user name	Required	Text
Excluded Ingredients	Allow users exclude unwanted ingredients	Required	Text
Save Settings	Confirms and saves updates	Optional	Button

Table 6: Profile and settings interface objects

Add Recipe Interface		
Component	Function	Type
Recipe name	It provides the name of the recipe	Text
Ingredients	This Lists all required ingredients	Text
Steps	Give preparation instructions	Text
Nutrition Info	Shows calories and nutrients	Number/text
Save Button	Adds to favorites	Button
Add to Planner	Schedules meal for a chosen day	Button

Table 7: Add recipe interface objects

Forgot Password Interface		
Input Field	Data Type	description
Email	Text	Used to identify account
New Password	Encrypted Text	Creates a new password
Reset Password	Button	Update the new password

Table 8: Forget password interface objects

Favorites Interface	
Item	Function
Remove Button	Deletes recipe from favorites
Open Button	Opens recipe details page
Recipe List	Displays all saved recipes we have

Table 9: Favorites interface objects

Edit Recipe Interface		
Item	Function	Type
Recipe Image	Change recipe image	Image
Title	Edit recipe name	Text
Ingredients	Edit ingredients	List
Steps	Edit instructions	Text
Save	Save changes	Button

Table 10: Edit recipe interface objects

Plan Weekly Meals Interface		
Item	Function	Type
Calendar	Select day	Selector
Recipe List	Choose recipe	List
Meal Type	Select meal	Toggle
Save Meal	Add meal to plan	Button
Success Message	Confirm saved plan	Text

Table 11: Plan weekly meals interface objects

Personalized Meal Recommendations Interface		
Item	Function	Type
Recipe Card	Display recommended meal	Card
Image	Show meal picture	Image
Title	Show meal name	Text
Like	Add to liked recipes	Icon
Save	Add to saved recipes	Icon

Table 12: Personalized meal recommendations interface objects

4.5 Other Interfaces

Sign-up Interface:

The registration interface will display messages to the user, including that if they do not enter their name, it will notify them that it is required, or if their email address already exists or is previously registered. To confirm the password, if it does not match, it will also show an error message. A screen will appear if the registration is completed successfully.

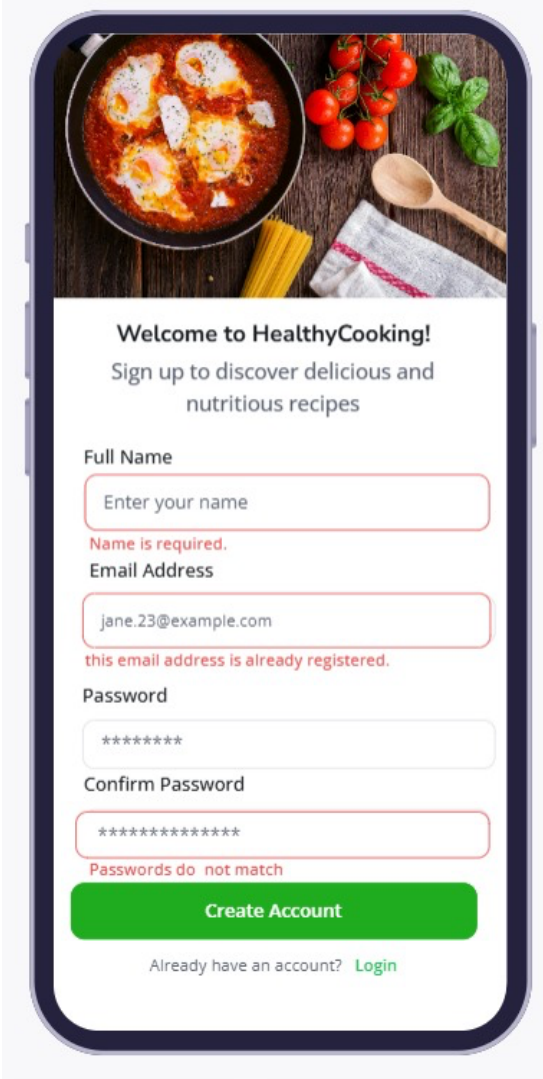


Figure 22 shows a mobile app sign-up interface. At the top is a header image of a pan with food. Below it, the text "Welcome to HealthyCooking!" is displayed, followed by "Sign up to discover delicious and nutritious recipes". The form consists of four input fields: "Full Name" (with a red border and the message "Name is required."), "Email Address" (with a red border and the message "this email address is already registered."), "Password" (with a red border and the message "Passwords do not match"), and "Confirm Password" (with a red border and the message "Passwords do not match"). A green "Create Account" button is at the bottom, and a link "Already have an account? Login" is below it.

Figure 22: Additional sign-up interface

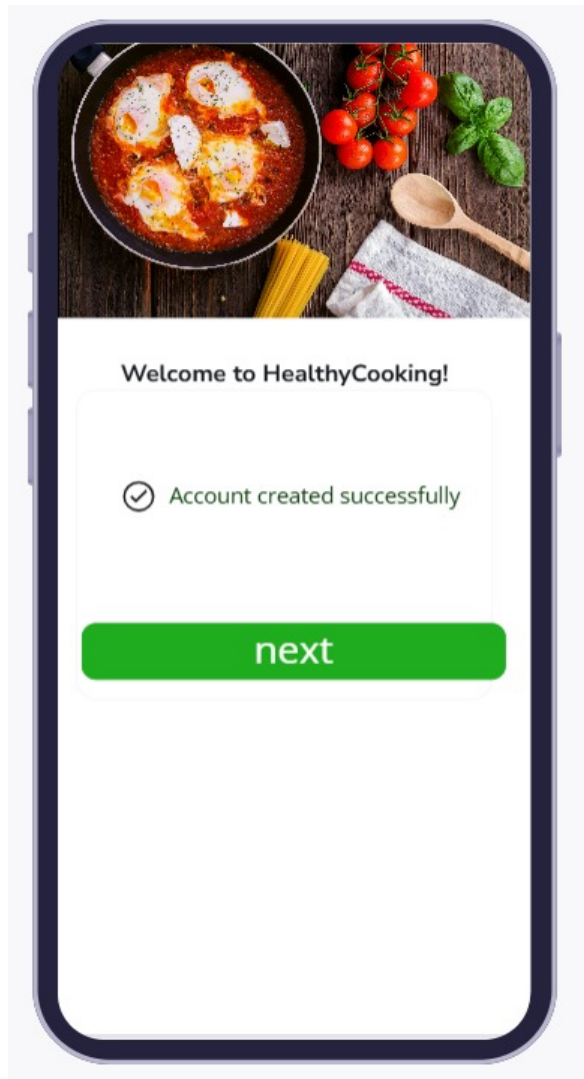


Figure 23 shows a mobile app sign-up interface. At the top is a header image of a pan with food. Below it, the text "Welcome to HealthyCooking!" is displayed, followed by "Sign up to discover delicious and nutritious recipes". A green checkmark icon is shown next to the text "Account created successfully". A green "next" button is at the bottom.

Figure 23: Additional sign-up interface

Forgot Password Interface:

When a new password is set for the user and the email is confirmed, it will give a message that the password has been successfully confirmed and updated.

Figure 24: Additional forget password interface

Log-in Interface:

In the Log-in interface, the system interacts with the authentication service to validate the user's email and password. If the entered information is incorrect, an error message is displayed to notify the user and restrict access.

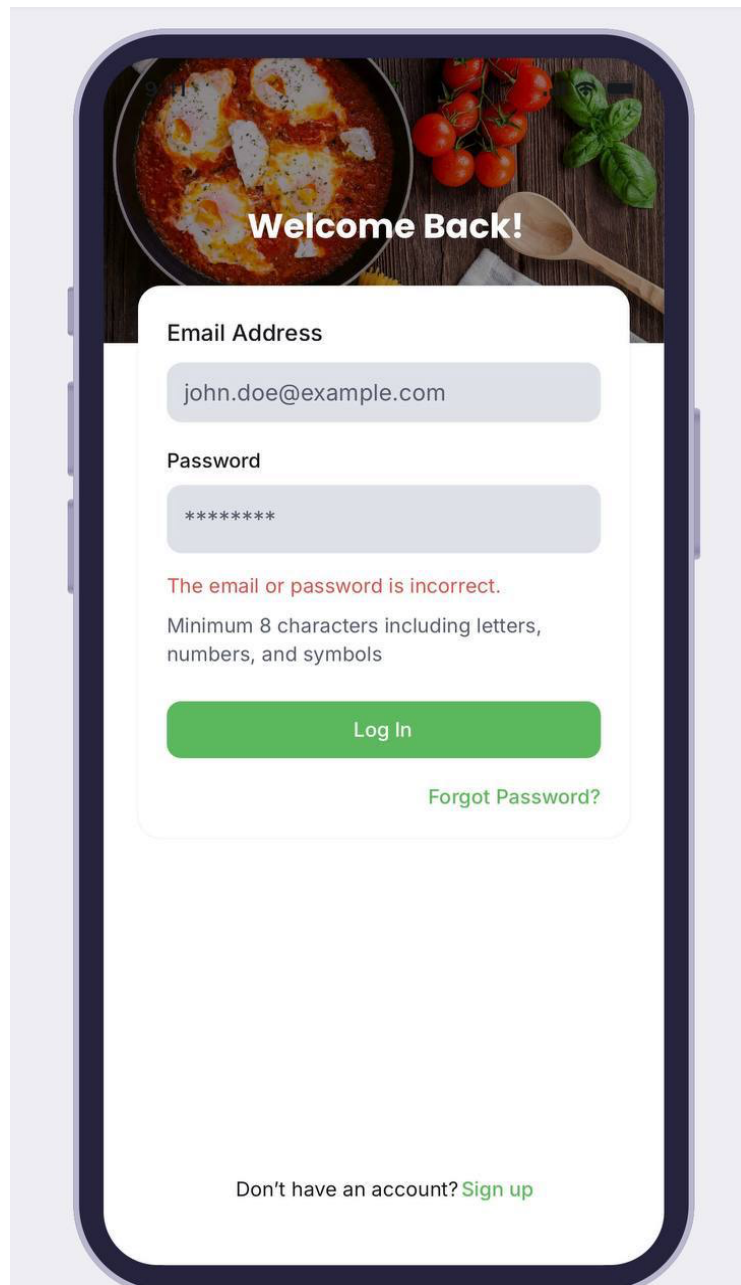


Figure 25: Additional log-in interface

Profile Management Interface:

The profile page will display a message requiring the name and the addition of allergens or some ingredients that are not preferred by the user, or to write that there are none. Upon completion, it will display a message confirming the successful saving of the updated data.

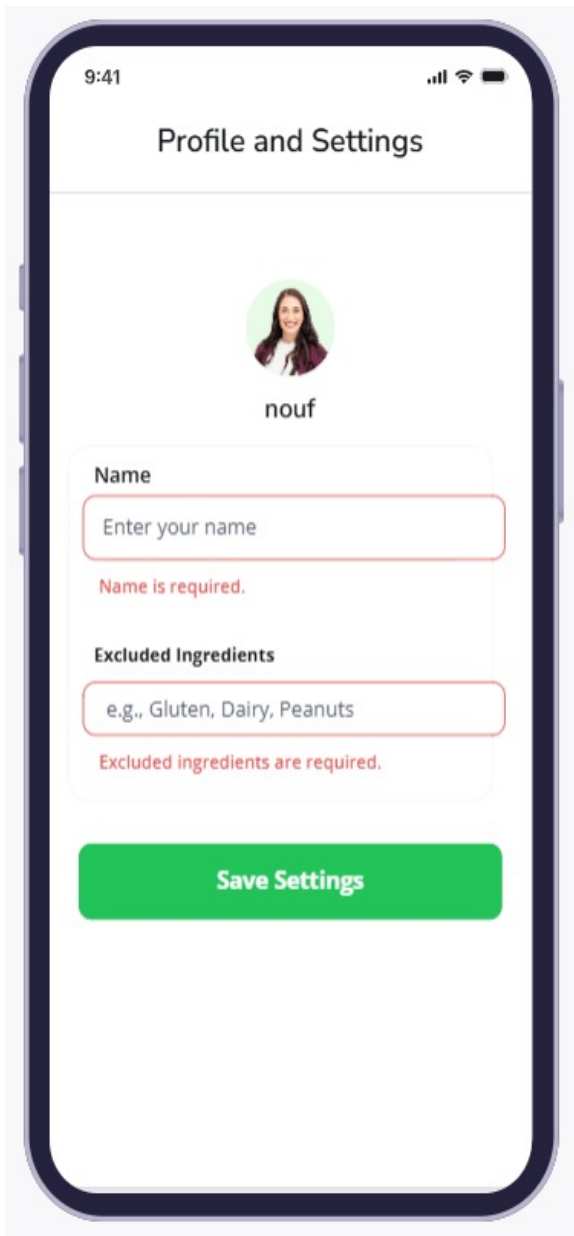


Figure 27: Additional profile management interface

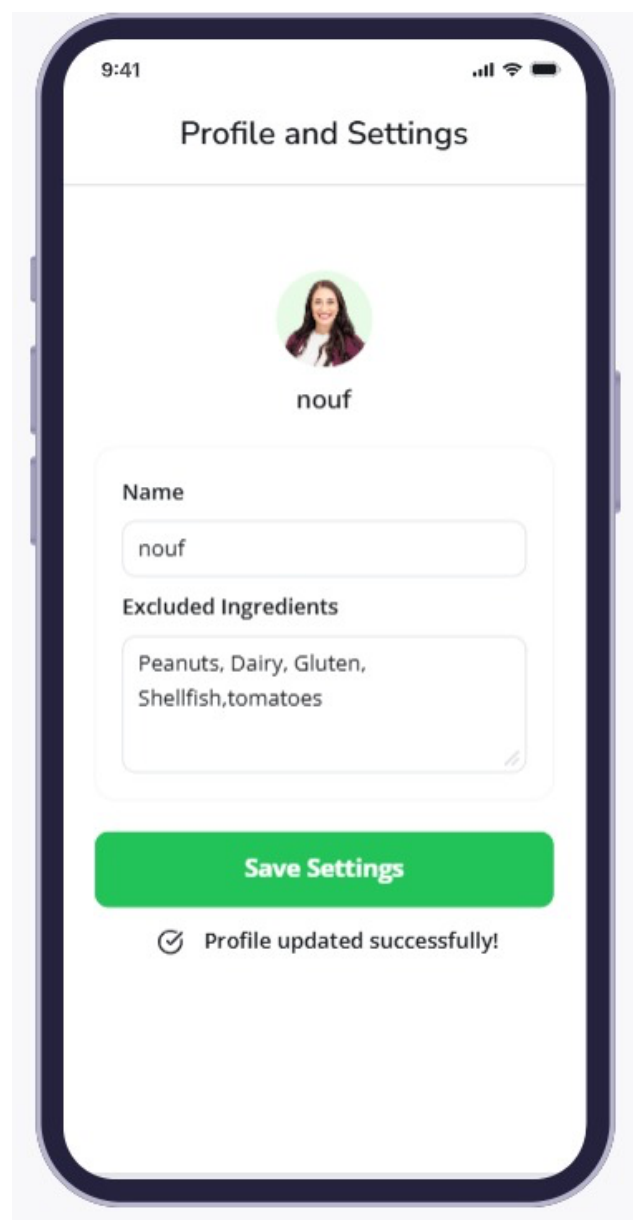


Figure 26: Additional profile management interface

Delete Recipe Interface:

When you press the remove button to delete the recipe from favorites, a "Deleted successfully" page will appear for confirmation, but here a "Nothing has been removed" page will appear, indicating that the recipe has not been deleted from favorites.

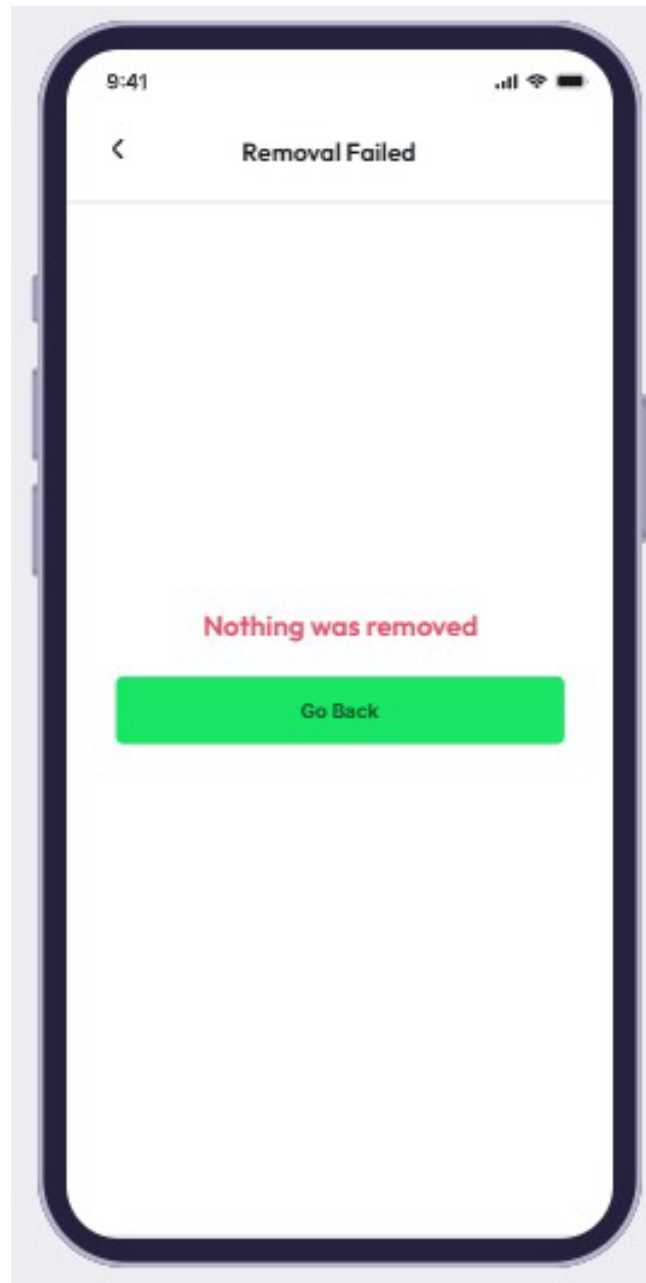


Figure 28: Additional delete recipe interface

5. System Architecture

This section provides an overview of how the HealthyCooking system functionalities and responsibilities are partitioned into a layered architecture. It also shows how the main components are decomposed, how they interact with each other, and why this architecture was chosen to keep the system simple, scalable, and easy to maintain.

5.1 Architectural Design Approach

The most suitable architectural design approach for the HealthyCooking system is the layered architecture approach, as it works effectively for both iOS and Android mobile applications. Each system function is placed in its appropriate layer, so any issue or maintain processes will only affect that specific layer. This architecture makes the system easier to maintain, more secure, and less independent.

Other architectural approaches were considered but not selected because they do not provide the same level of modularity as the chosen layered architecture. Selecting non-layered approach would make future updates harder to manage, which also may negatively impact the security and scalability of the system.

The following figure illustrates the 3 layers architecture of the HealthyCooking system application.

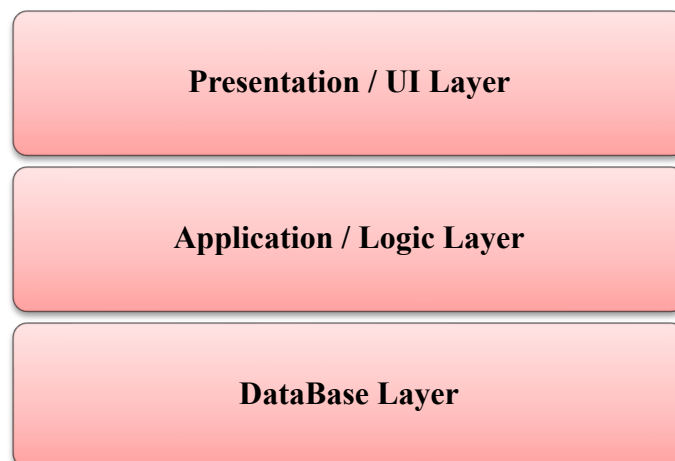


Figure 29: The HealthyCooking system architecture design approach

5.2 Architectural Design

A description of the HealthyCooking system application architecture will be discussed and illustrated with figure below in this subsection.

1. Presentation / UI Layer:

The first layer of the system that is responsible for all end-user interactions. It displays all information that user needs to have, then it takes the user inputs and passes them to the next layer. SwiftUI for iOS and XML for Android will be used to build the User Interface of this upper layer.

2. Application / Logic Layer:

The second layer of the system that is responsible for performing all application's functions, including authentication, profile management, recipes, and weekly meal planning. Swift for iOS and Kotlin for Android programming languages will be used to build this layer of the system.

3. DataBase Layer:

The third layer of the system that is responsible for storing, retrieving, updating, and deleting all system data securely. SQL based-database will be used to build this lowest layer of the system.

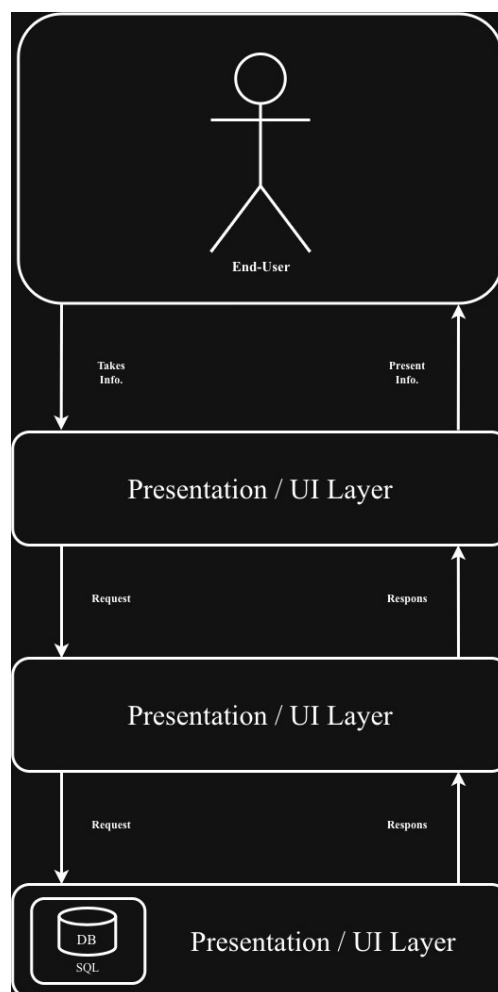


Figure 30: The HealthyCooking system design architecture

5.3 Subsystem Architecture

In the following DFD diagrams are used to represent the whole and subsystems architecture of the HealthyCooking system. Below, an illustration of HealthyCooking context DFD diagram is shown, as it has the end-user, whole system, and external APIs working together.

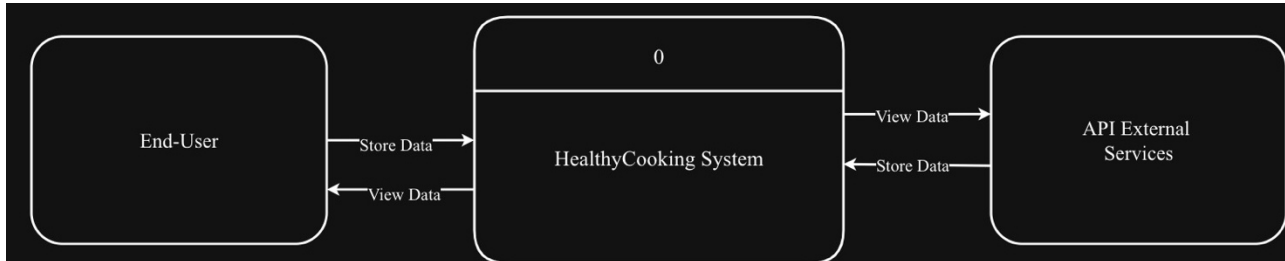


Figure 31: Context DFD of HealthyCooking system

5.3.1 User Authentication Subsystem

In the user authentication subsystem, there are four functionalities that work with the user table in the database, along with the user. The functionalities are sign-up, log-in, rest password, and log-out; as shown in the figure below.

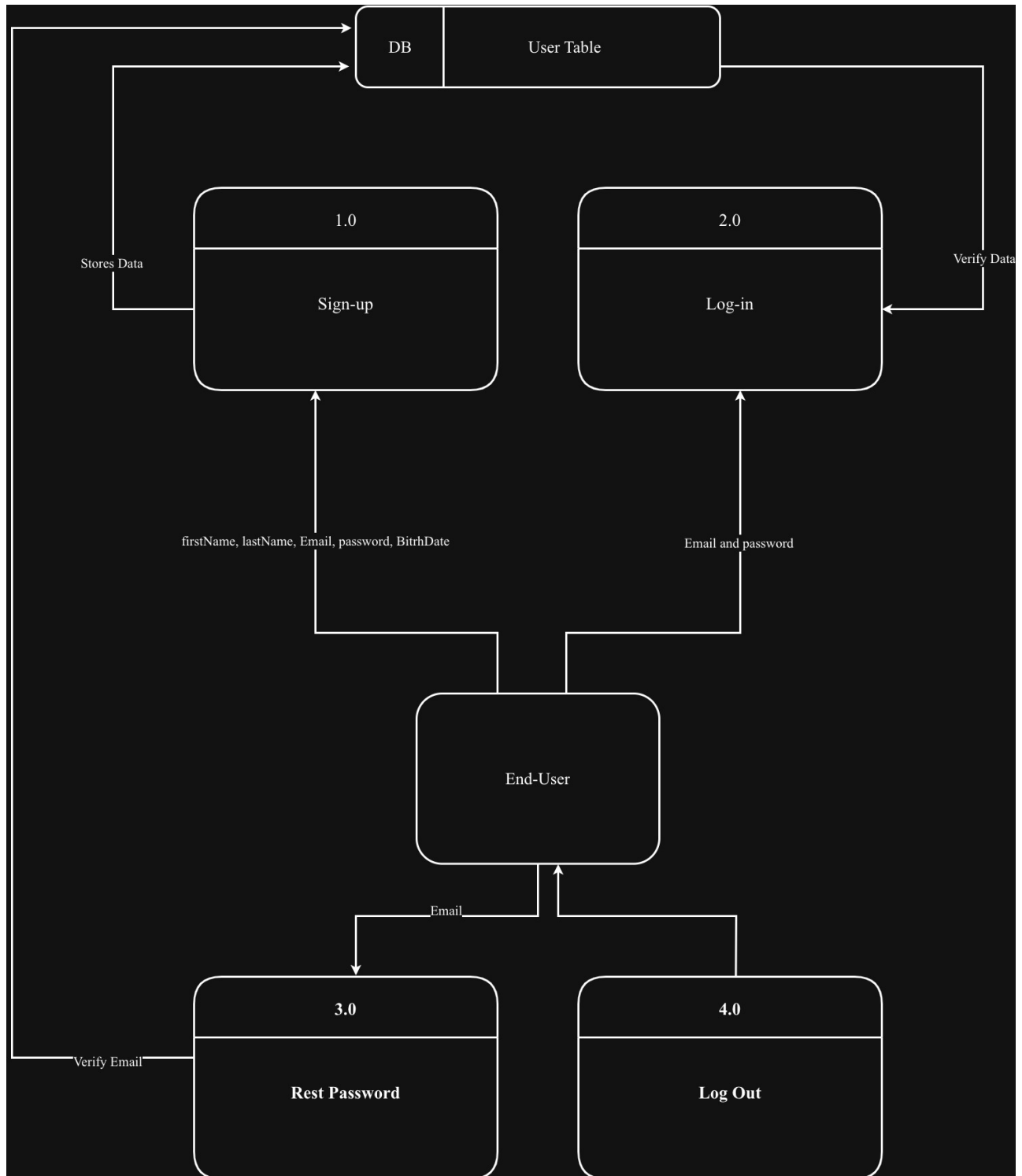


Figure 32: User authentication subsystem functionalities

5.3.2 User Profile Subsystem

In the user profile subsystem, there are two functionalities that work with the user table in the database, along with the use. The functionalities are update and view profile. Update profile info enables user to edit the info they entered when they signed up and add more info as their weight and health goal. View profile service views all user sign-up info, plus user weight and health goal if they exist; as shown in the figure below.

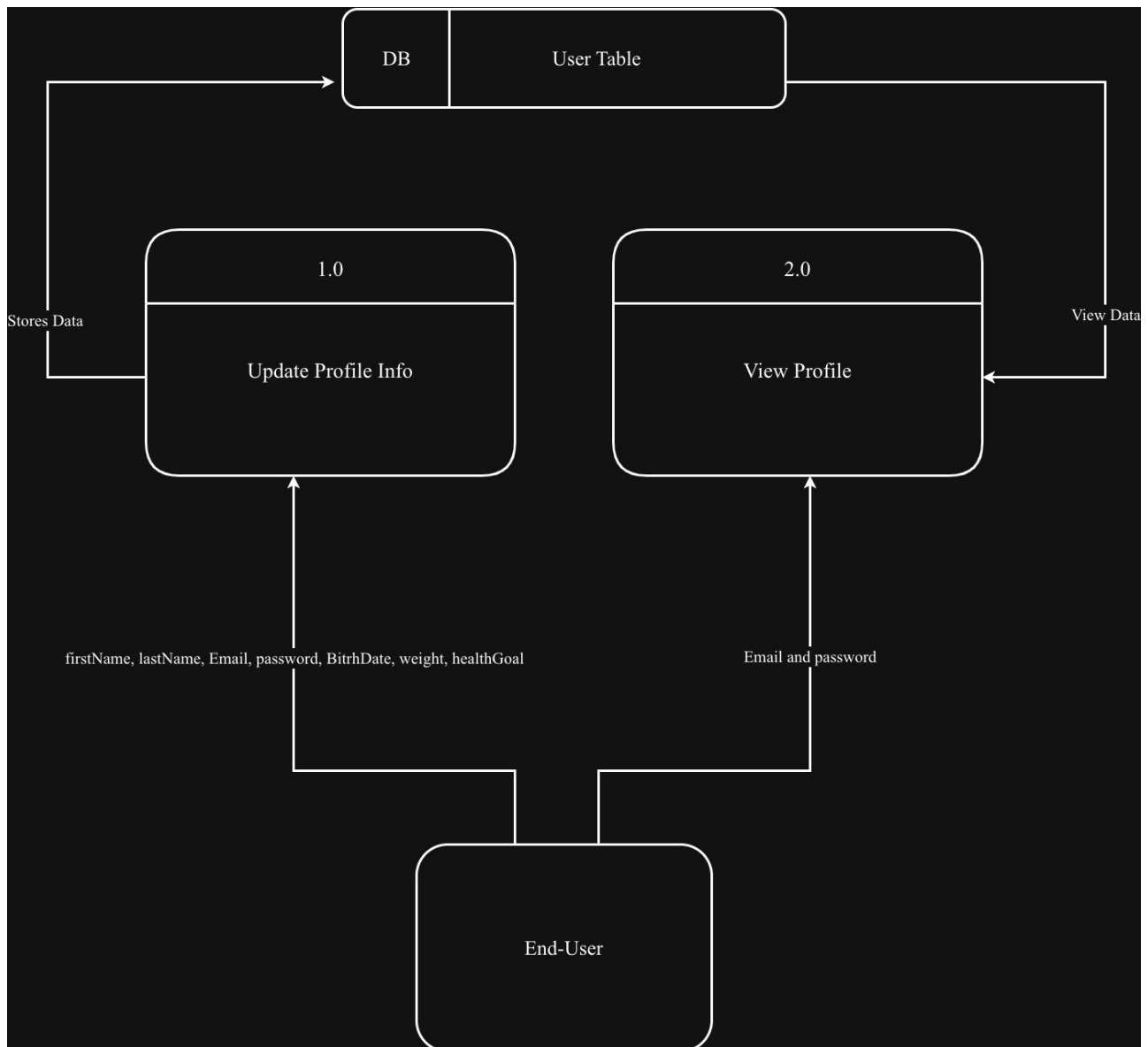


Figure 33: User profile subsystem functionalities

5.3.3 Recipe Management Subsystem

In the recipe management subsystem, there are five functionalities that work with the recipe table in the database, along with the end user, as shown in the figure below. The functionalities are add, update, delete, view, search recipe. End user enters required info for the function they want and then the system communicates with the recipe table in the database and do the needed work and store things successfully.

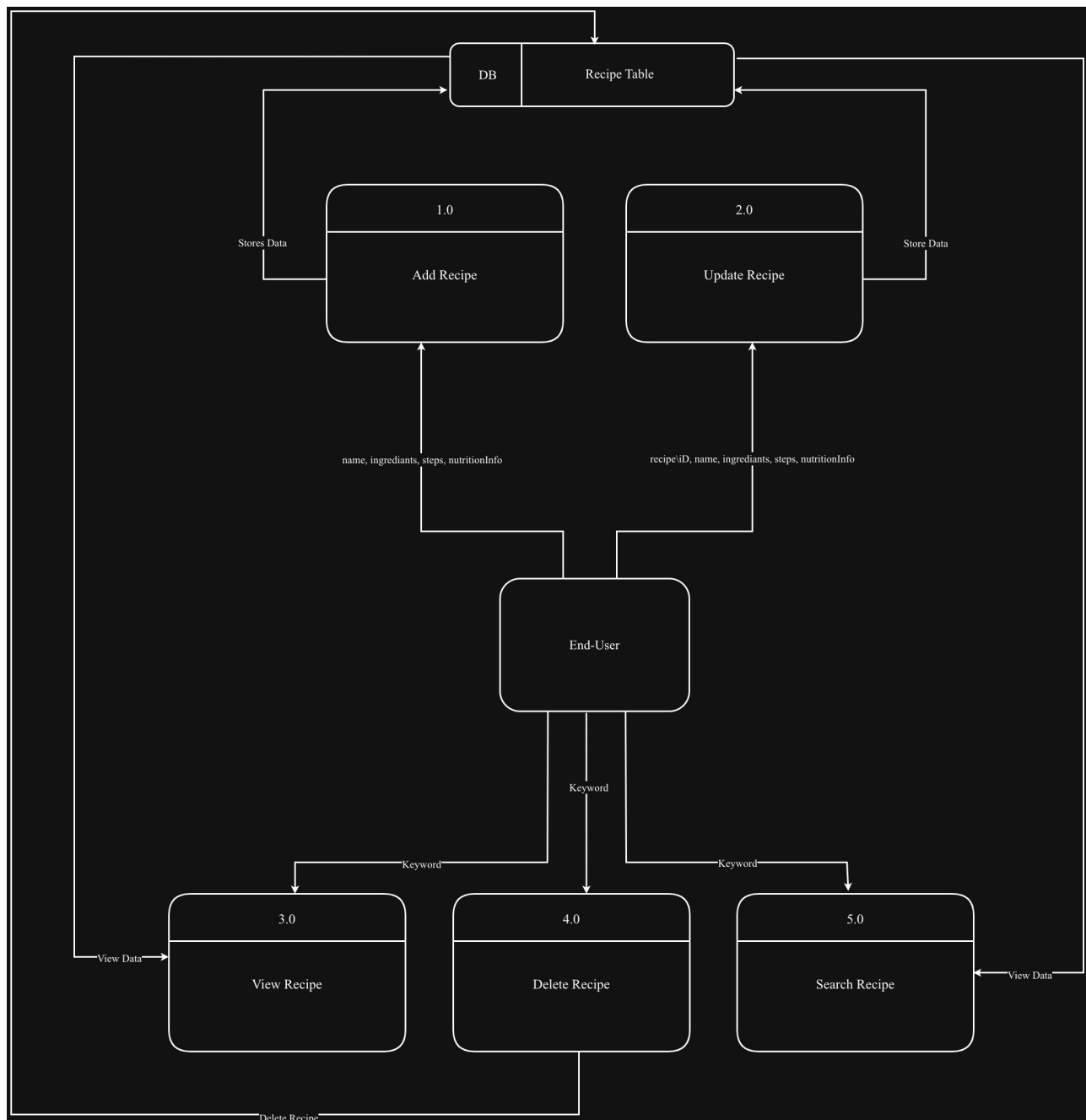


Figure 34: Recipe management subsystem functionalities

5.3.4 Meal Planner Subsystem

In the meal planner subsystem, there are two functionalities that work with the week plan table in the database along with the end user, as shown in the figure below. The functionalities are generate and view weekly plan. End user should select and schedule meals and then the system should communicate with the week plan table and do the needed work and store things successfully.

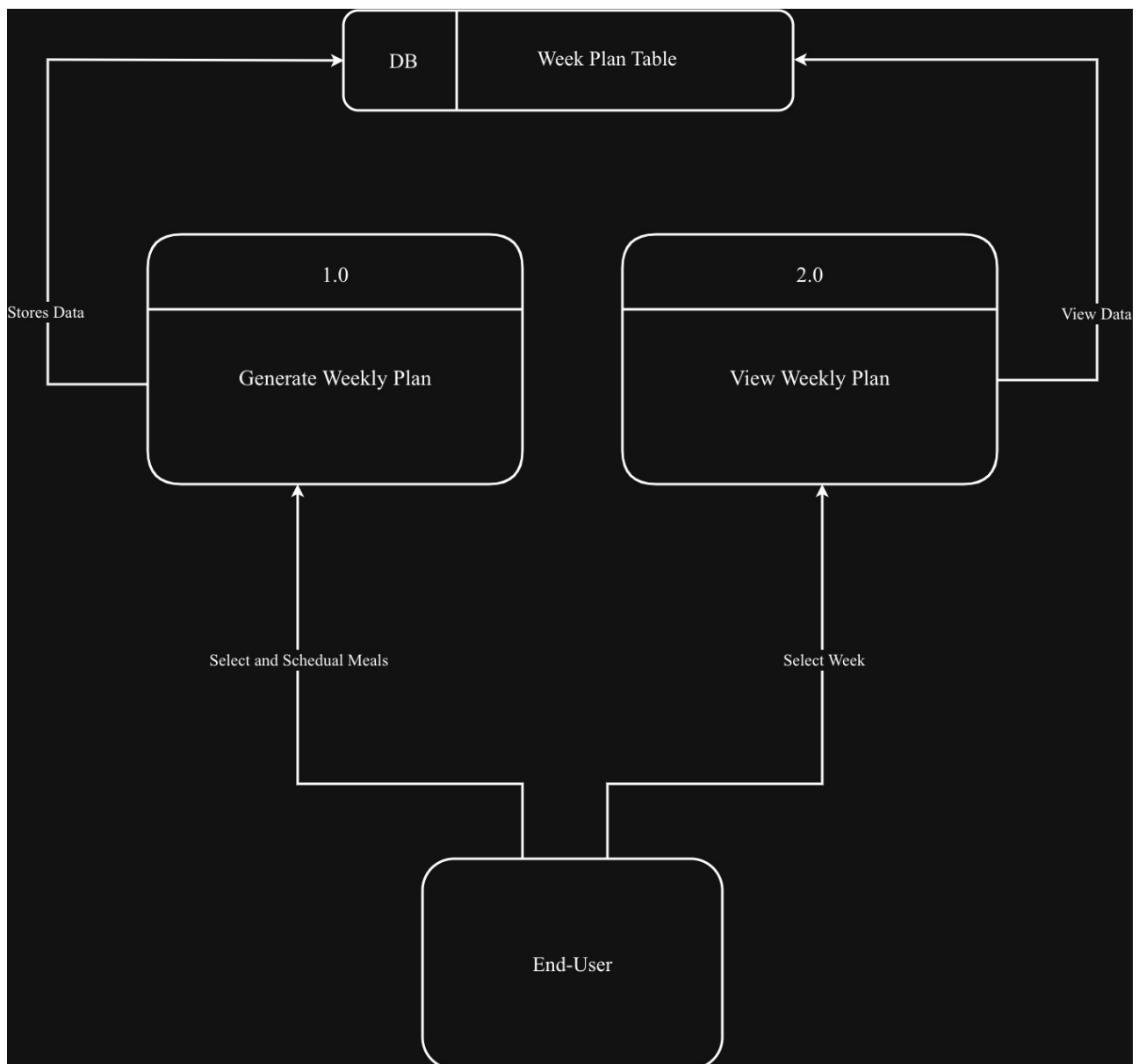


Figure 35: Meal planner subsystem functionalities

5.3.5 Calorie Tracker Subsystem

In the calorie tracker subsystem, there are three functionalities that work with the user and user meals tables in the database, along with the end user. As shown below in the figure, the functionalities are add meal, calculate daily calories, and view daily summary. Add meal service enables the user to add meal they take to add its calories and nutrition info to the database. Calculate daily calories, calculates the suitable amount of calories to the user based on their information that are already exist in the database. View daily summary service shows the calories of the day and calculates the remaining calories. End user should choose and enter the data they want and then the system should communicate with the database and do the needed work and store things successfully.

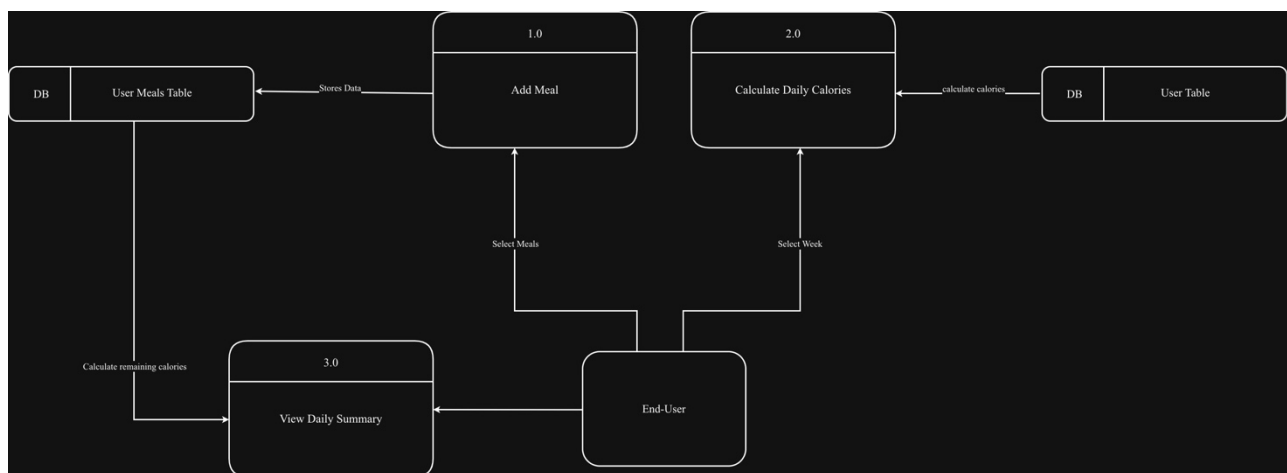


Figure 36: Calorie tracker subsystem functionalities

6. Data Design

In this data design section, a description of how HealthyCooking system data are stored and transformed into the database. Also, a detail description of data entities, fields, types, and constraints will be shown in tables, schema, and ER diagram.

6.1 Data Description

Information of the HealthyCooking system provided by the end-user will be stored in the system database in a structured way. Major entities such as User, Recipe, WeekPlan, and UserMeals are represented as SQL tables with clear fields and relationships to ensure data integrity and flexibility. The data is organized through primary and foreign keys that link the entities together. Each end-user can access and update only their own data, while the rest of the system remains protected from unauthorized modifications.

6.2 Data Dictionary

Each entity in the system must have its fields, and each field should be of specific type. Each field may be a primary or foreign key and may have constraints as shown in the table below. Each entity has relationships with other entities in the system and that would be shown in the later sub-section using an ER diagram and database schema. Also, all entity fields description are provided in the 7th section of this document.

Entity	Field	Type	PK	FK	Constraints
User	UserID	INT(10)	YES	YES	NOT NULL
	firstName	VARCHAR(30)			NOT NULL
	lastName	VARCHAR(30)			
	Email	VARCHAR(30)		YES	NOT NULL
	password	VARCHAR(30)			NOT NULL
	birthDate	DATE			
	weight	INT(3)			
	healthGoal	INT(3)			
	dailyCalorites	INT(4)		YES	
Recipe	RecipeID	INT(10)	YES	YES	NOT NULL
	recipeName	VARCHAR(30)			NOT NULL
	ingredients	VARCHAR(50)			NOT NULL
	steps	VARCHAR(100)			NOT NULL
	nutritionInfo	VARCHAR(30)			
	category	VARCHAR(30)			NOT NULL
WeekPlan	meal	VARCHAR(30)			
	scheduledDay	VARCHAR(10)			
UserMeals	meals	VARCHAR(30)			
	remainingCalories	VARCHAR(4)			

Table 13: Data description of HealthyCooking system

6.3 Database Description

The HealthyCooking system uses a relational SQL database to store and manage system entities. The database is structured into four main tables: User, Recipe, UserMeals, and WeekPlan.

The User table stores authentication data, daily calorie goals, and personal health preferences.

The Recipe table contains recipe information including name, ingredients, steps, and nutritional info.

The UserMeals table represents the many-to-many relationship between users and recipes and stores daily nutrition tracking information.

The WeekPlan table stores the user's weekly meal schedule by connecting a user to a selected recipe for a specific day.

The primary keys, UserID, RecipeID uniquely identify each record, while foreign keys maintain referential integrity between tables.

The two figures below respectively shows the complete ER diagram and the relational database schema, including all entities, attributes, and relationships.

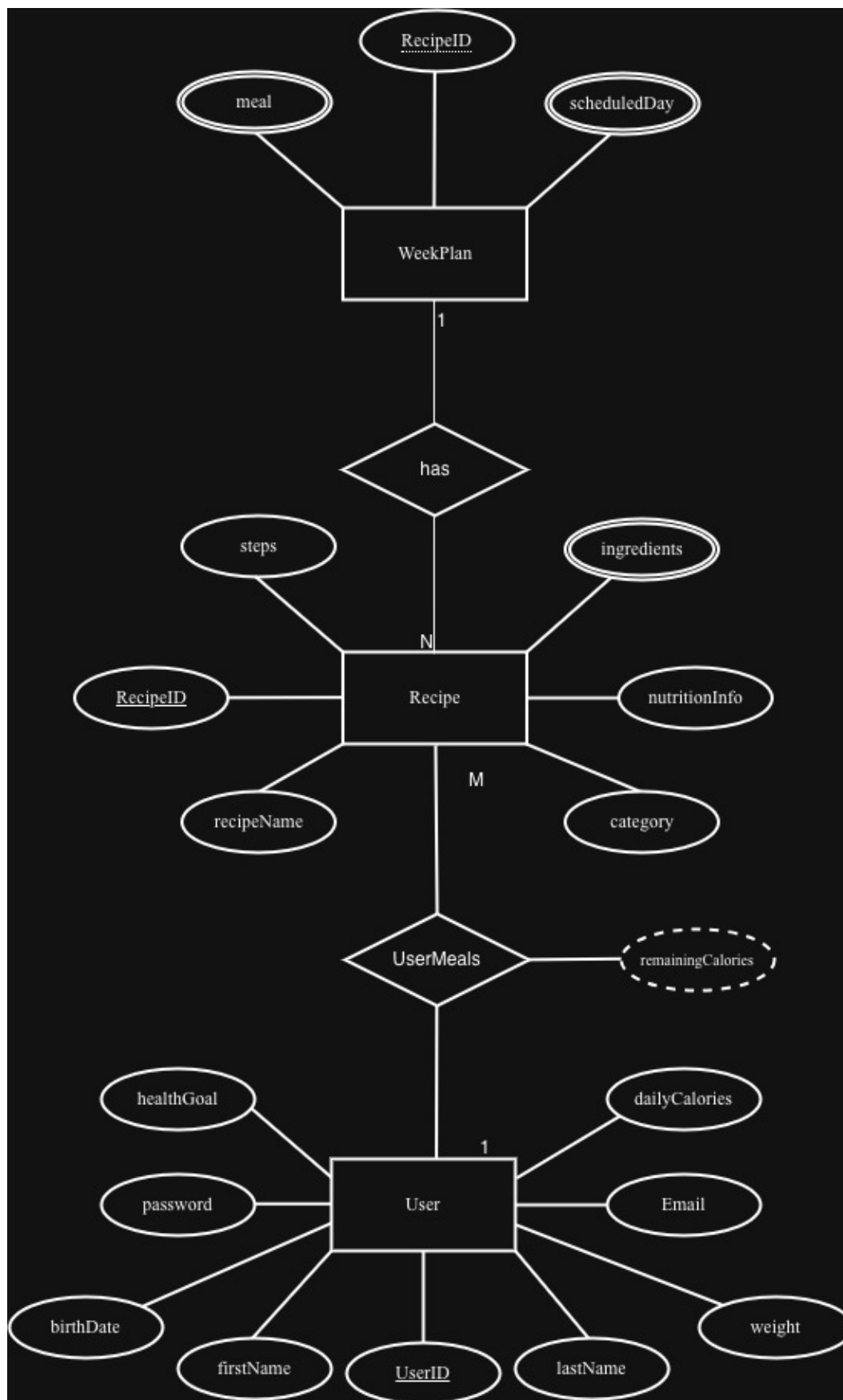


Figure 37: HealthyCooking ER Diagram

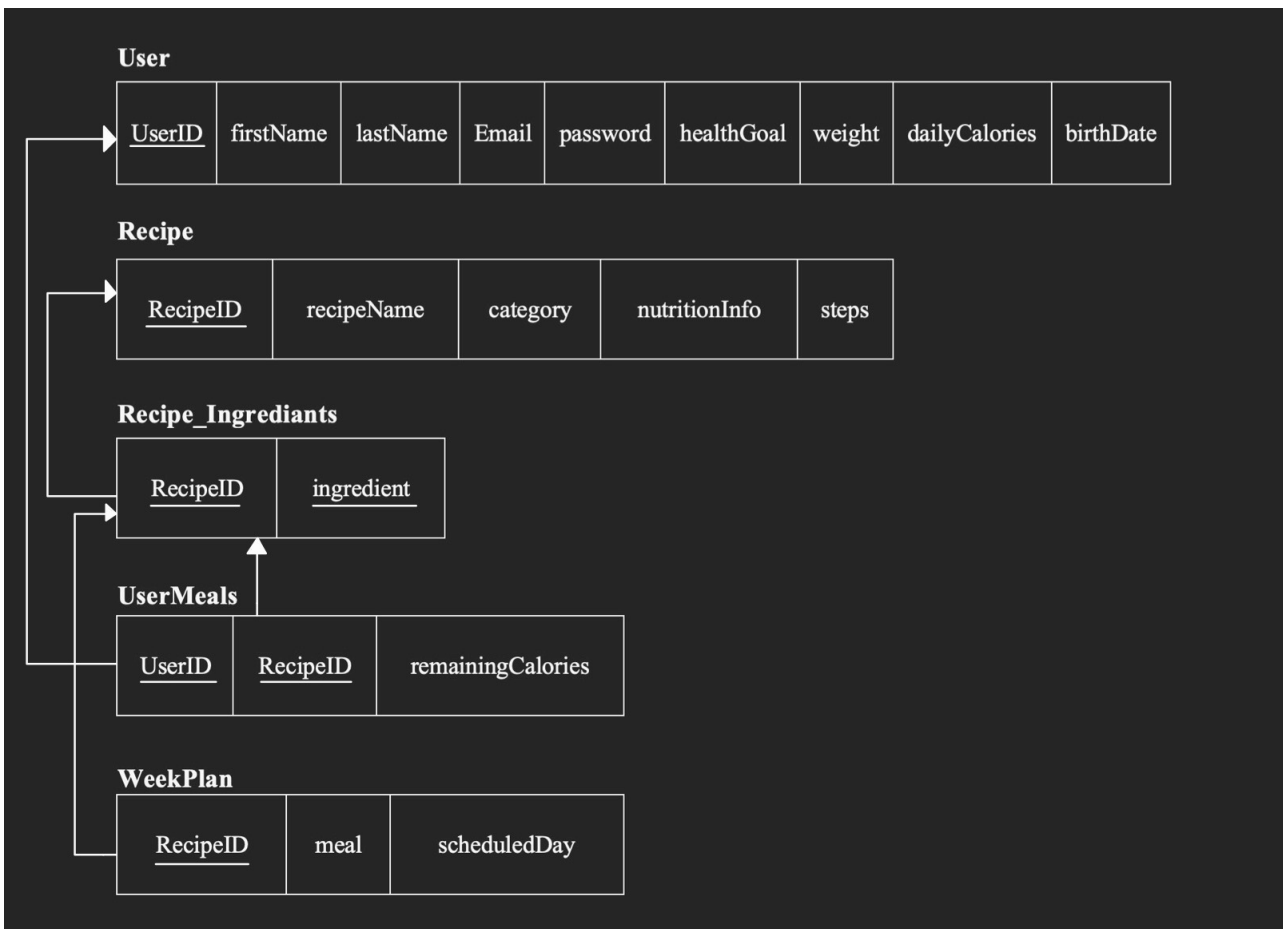


Figure 38: HealthyCooking Relational Schema

7. Component Design

In this section, we take a closer look at each major component in the HealthyCooking system. A description of the prime functionalities will be provided to describe the main functions of the application. Also, a pseudocode segments will be written in a natural language to give more details. And a list of local variables would be presented in tables.

7.1 User Authentication Component

The user authentication component manages users' sign-up, log-in, and log-out functionalities. It ensures that only authorized users access the system features. It communicates with the system database to save and verify the user inputs. The component also supports password encryption to maintain system security.

User Authentication Component Pseudocodes

The following figure illustrates the pseudocode of the sign-up function. As it takes inputs from user, it then checks if the email already exists in the database. If not, it stores the user inputs as a new record in the database of the HealthyCooking system.

```
FUNCTION sign-up(firstName, lastName, Email, password, BirthDate)
    IF userExists(Email) THEN
        DISPLAY "User already exists"
    ELSE
        addUserToDatabase(firstName, lastName, Email, password, BirthDate)
        DISPLAY "Registration successful"
    ENDIF
END FUNCTION
```

Figure 39: pseudocode of the sign-up function

The following figure illustrates the pseudocode of the log-in function. As it takes Email and password as inputs from the user, then checks if the email exists in the database and the entered password matches the one stored in the database. If yes, it will grant user access to the system.

```
FUNCTION login(Email, password)  
    user = findUserInDatabase(Email)  
    IF user EXISTS AND user.password == password THEN  
        DISPLAY "Login successful"  
        RETURN TRUE  
    ELSE  
        DISPLAY "Invalid Email or password, try again."  
        RETURN FALSE  
    ENDIF  
END FUNCTION
```

Figure 40: pseudocode of the log-in function

The following figure illustrates the pseudocode of the resetPassword function. As it takes an Email and a new password as inputs from the user, then updates the password if the Email exists in the database.

```
FUNCTION resetPassword(Email, newPassword)  
    IF userExists(Email) THEN  
        updatePasswordInDatabase(Email, newPassword)  
        DISPLAY "Password updated successfully"  
    ELSE  
        DISPLAY "Email not found"  
    ENDIF  
END FUNCTION
```

Figure 41: pseudocode of the reset password function

The following figure illustrates the pseudocode of the log-out function.

```
FUNCTION logout()  
    DISPLAY "User logged out"  
END FUNCTION
```

Figure 42: pseudocode of the log-out function

User Authentication Component Local Data

The local data of the authentication component is described in detail in the table below.

Local Variable	Description
firstName	The first name of the user provided during registration.
lastName	The last name of the user provided during registration.
Email	User's Email address used for sign-up, login, or password reset.
password	Password entered by the user for authentication
birthDate	The user's date of birth, entered during registration, used for age-based calculations.
newPassword	The new password entered by the user when performing a password reset.

Table 14: Local data of user authentication Component

7.2 User Profile Component

The user profile component is responsible for managing the user profile in the HealthyCooking system. It provides update and view profile functionalities, ensuring that the system stores and views the correct data that the user provides for their profile, by accessing the database of the application.

User Profile Component Pseudocodes

The following figure illustrates the pseudocode of the updateProfile function. As it takes firstName, lastName, Email, birthdate, weight, and healthGoal as inputs from the user then updates the profile, if the Email exists in the database.

```
FUNCTION updateProfile(Email, firstName, lastName, birthDate, weight, healthGoal)
    IF userExists(Email) THEN
        updateUserInDatabase(Email, firstName, lastName, birthDate, weight, healthGoal)
        DISPLAY "Profile updated successfully"
    ELSE
        DISPLAY "User not found"
    ENDIF
END FUNCTION
```

Figure 43: pseudocode of the update profile function

The following figure illustrates the pseudocode of the viewProfile function. As it takes an Email as an input argument from the user, it checks if the Email exists in the system database. If yes, it displays Email, firstName, lastName, birthdate, weight, and healthGoal of that user.

```
FUNCTION viewProfile(Email)
    user = getUserFromDatabase(Email)
    DISPLAY user.Email, user.firstName, user.lastName,
    user.birthDate, user.weight, user.healthGoal
END FUNCTION
```

Figure 44: pseudocode of the view profile function

User Profile Component Local Data

The local data of the profile component are described in detail in the table below.

Local Variable	Description
Email	User's Email address provided during registration.
firstName	The first name of the user provided during registration.
lastName	The last name of the user provided during registration.
birthDate	User's birth date provided during registration, that is used for age-based calculations.
weight	User's current weight.
healthGoal	User's health goal (lose, maintain, or gain weight).

Table 15: local data of the user profile component

7.3 Recipe Management Components

The recipe management component is responsible for managing the recipes in the system database. It provides add, edit, delete, and view recipe functionalities, ensuring that the system deals with recipes in a correct way, by accessing the database of the application.

Recipe Management Component Pseudocodes

The following figure illustrates the pseudocode of the addRecipe function. As it takes recipe information from the user, such as name, ingredients, steps, nutritionInfo, and stores it in the HealthyCooking database system.

```
FUNCTION addRecipe(name, ingredients, steps, nutritionInfo)
    insertRecipeIntoDatabase(name, ingredients, steps, nutritionInfo)
    DISPLAY "Recipe added successfully"
ENDIF
END FUNCTION
```

Figure 45: pseudocode of the add recipe function

The following figure illustrates the pseudocode of the updateRecipe function. As it takes recipe information from the user, such as name, ingredients, steps, nutritionInfo as arguments, then checks if the recipe exists in the database, if yes, it updates recipe information with the provided arguments.

```
FUNCTION updateRecipe(recipeID, name, ingredients, steps, nutritionInfo)
    IF recipeExistsByID(recipeID) THEN
        updateRecipeInDatabase(recipeID, name, ingredients, steps, nutritionInfo)
        DISPLAY "Recipe updated successfully"
    ELSE
        DISPLAY "Recipe not found"
    ENDIF
END FUNCTION
```

Figure 46: pseudocode of the update recipe function

The following figure illustrates the pseudocode of the deleteRecipe function. As it lets the user choose one of the recipes, they add to takes recipe information from the user, such as name, ingredients, steps, nutritionInfo as arguments, then it deletes it from the system database by the recipeID.

```
FUNCTION deleteRecipe(recipeID)  
  IF recipeExistsByID(recipeID) THEN  
    removeRecipeFromDatabase(recipeID)  
    DISPLAY "Recipe deleted successfully"  
  ELSE  
    DISPLAY "Recipe not found"  
  ENDIF  
END FUNCTION
```

Figure 47: pseudocode of the delete recipe function

The following figure illustrates the pseudocode of the viewRecipe function. As it lets the user choose one of the recipes from the system database, it displays its information, such as its name, ingredients, steps, and nutrition info.

```
FUNCTION viewRecipe(recipeID)  
  recipe = getRecipeFromDatabase(recipeID)  
  DISPLAY recipe.name, recipe.ingredients,  
  recipe.steps, recipe.nutritionInfo  
END FUNCTION
```

Figure 48: pseudocode of the view recipe function

The following figure illustrates the pseudocode of the SearchRecipe function. As it creates an empty list at first to save the results in it. Then it searches for any recipe name that matches the keyword that the user entered, or any recipe with a matching category or ingredient. Then it stores them in the result list and return them to the user.

FUNCTION SearchRecipe(keyword)

```
    resultList ← empty list
    recipe ← temporary variable for looping

    FOR each recipe IN RecipeDatabase DO
        IF keyword matches recipe.name OR
           keyword matches recipe.category OR
           keyword matches any ingredient in recipe.ingredients
        THEN
            ADD recipe TO resultList
        ENDIF
    ENDFOR

    RETURN resultList

END FUNCTION
```

Figure 49: pseudocode of the search recipe function

Recipe Management Component Local Data

The local data of the recipe component are described in detail in the table below.

Local Variable	Description
recipeID	Recipe's ID provided by the system in the database.
name	Recipe's name provided by the user.
ingredients	Recipe's ingredients provided by the user.
steps	Recipe's steps provided by the user.
nutritionInfo	Recipe's nutrition info provided by the user.
resultList	A list that stores all recipes returned from the search.
recipe	A single recipe object containing its data.
category	The classification of the recip.

Table 16: Local data of recipe management component

7.4 Meal Planner Component

The meal planner component is responsible for managing the recipes in a weekly schedule, it also ensures that the user's goal and calorie limits are considered. It provides generate and view weekly plan functionalities and deals with the HealthyCooking database system.

Meal Planner Component Pseudocodes

The following figure illustrates the pseudocode of the `generateWeeklyPlan` function. As it calculates the user's calories that the user should maintain, based on the information that the system has about the user. Then it lets user schedule recipes from the system database based on their preferences. After that, it saves the user's weekly plan in the database.

```
FUNCTION generateWeeklyPlan(userID)
    user = getUserFromDatabase(userID)
    dailyCalories = calculateDailyCalories(user.age, user.weight,
user.healthGoal)
    weekPlan = EMPTY LIST

    FOR day IN 1 TO 7
        meals = selectMealsForDay(dailyCalories, user.preferences)
        weekPlan[day] = meals
    END FOR

    saveWeeklyPlanToDatabase(userID, weekPlan)
    DISPLAY "Weekly meal plan generated successfully"
END FUNCTION
```

Figure 50: pseudocode of the generate weekly plan function

The following figure illustrates the pseudocode of the `viewWeeklyPlan` function. As it previews the week plan that user has made previously.

```
FUNCTION viewWeeklyPlan(userID)
    weekPlan = getWeeklyPlanFromDatabase(userID)
    DISPLAY weekPlan
END FUNCTION
```

Figure 51: pseudocode of the view weekly plan function

Meal Planner Component Local Data

The local data of the recipe component are described in detail in the table below.

Local Variable	Description
userID	User's ID provided by the system in the database.
age	Calculated user age, derived from the birth date.
weight	User's current weight.
healthGoal	User's health goal (lose, maintain, or gain weight).
dailyCalories	Calculated target calories per day for the user.

Table 17: Local data of meal planner component

7.5 Calorie Tracker Component

The user profile component is responsible for tracking each user's calories and storing them in the HealthyCooking database. It provides addMeal, calculateDailyCalories, and viewDailySummary functionalities, ensuring that the system stores maintain user's calorie intake information.

Calorie Tracker Component Pseudocodes

The following figure illustrates the pseudocode of the addMeal function. As it calculates the amount of calories of the meal that the user has chosen.

```
FUNCTION addMeal(userID, mealID)  
    meal = getMealFromDatabase(mealID)  
    logMealForUser(userID, meal)  
    DISPLAY "Meal added successfully"  
END FUNCTION
```

Figure 52: pseudocode of the add meal function

The following figure illustrates the pseudocode of the calculateDailyCalories function. As it calculates the total number of calories of the meals that the user has taken.

```
FUNCTION calculateDailyCalories(userID)  
    meals = getMealsLoggedForUser(userID,  
today)  
    totalCalories = 0  
  
    FOR each meal IN meals  
        totalCalories = totalCalories +  
meal.calories  
    END FOR  
  
    RETURN totalCalories  
END FUNCTION
```

Figure 53: pseudocode of the calculate daily calorie's function

The following figure illustrates the pseudocode of the viewDailyCalories function. As it shows users the calculations of their consumed and goal calories.

```
FUNCTION viewDailySummary(userID)
    totalCalories = calculateDailyCalories(userID)
    dailyGoal = getDailyCaloriesGoal(userID)
    DISPLAY "Total calories consumed: ", totalCalories
    DISPLAY "Daily calorie goal: ", dailyGoal
END FUNCTION
```

Figure 54: pseudocode of the view daily summary function

Calorie Tracker Component Local Data

The local data of the recipe component are described in detail in the table below.

Local Variable	Description
userID	User's ID provided by the system in the database.
mealID	Recipe's ID provided by the system in the database.
meal	Object storing meal data retrieved from the database.
meals	List of meals logged by the user for a specific day.
totalCalories	Sum of calories from all meals logged by the user.
dailyGoal	Daily calorie target for the user retrieved from the database.

Table 18: Local data of calorie tracker component

7.6 Notification Component

The notification component is responsible for sending meal remainders and calories alerts to notify the user. It has `sendMealReminder` and `sendCalorieAlert` functionalities ensuring that user gets notifications at their `mealTime`, and when they exceed their calculated calories limit.

Notification Component Pseudocodes

The following figure illustrates the pseudocode of the `sendMealReminder` function. As it sends messages to users when their `mealTime` arrives.

```
FUNCTION sendMealReminder(userID, mealTime)
  "!message = "It's time for your meal
  sendNotificationToUser(userID, message,
  mealTime)
  "DISPLAY "Meal reminder sent
END FUNCTION
```

Figure 55: pseudocode of the send meal reminder function

The following figure illustrates the pseudocode of the `sendCalorieAlert` function. As it sends notification alerts to users who exceeds their daily calorie's goal.

```
FUNCTION sendCalorieAlert(userID)
  totalCalories = calculateDailyCalories(userID)
  dailyGoal = getDailyCaloriesGoal(userID)
  IF totalCalories > dailyGoal THEN
    "!message = "You have exceeded your daily calorie goal
    sendNotificationToUser(userID, message)
    "DISPLAY "Calorie alert sent
  ENDIF
END FUNCTION
```

Figure 56: pseudocode of the send calorie alert function

Notification Component Local Data

The local data of the recipe component are described in detail in the table below.

Local Variable	Description
userID	User's ID provided by the system in the database.
mealTime	Scheduled time for meal reminder.
message	Notification message content to be sent to the user.
totalCalories	Sum of calories from all meals logged by the user.
dailyGoal	Daily calorie target for the user retrieved from the database.

Table 19: Local data of notification component

8. Detailed System Design

This section describes the internal design of the HealthyCooking system. It follows the same format and structure as the SDS Example, including classification, definition, uses and interactions, resources, processing, interface/exports, and system component diagrams.

8.1 Classification

This subsection classifies each component according to type and purpose.

Component Name	Component Type	Purpose
User Authentication Component	Common Component	Is responsible for handling the user account registration, login/logout, and password reset in case the user forgets
User Profile Component	User Component	Takes care and manages the users whole information including: dietary preferences, and calorie goals
Recipe Management Component	Functional Component	Manages recipe creation, modification, retrieval, and deletion
Weekly Meal Planner Component	Functional Component	Generates and manages the user's weekly meal plan
Calorie Tracker Component	Analytical Component	Calculates total daily calories and displays nutritional summaries
Notification Component	Service Component	Sends reminders and alerts related to meals and calorie goals

Table 20: Components classifications

8.2 Definition

This subsection defines the system components and describes their roles.

Component Name	Description
User Authentication Component	Verifies user credentials, manages secure access, and handles password reset.
User Profile Component	Stores and updates user information, dietary restrictions, and calorie goals.
Recipe Management Component	Monitoring and managing recipe's data, which includes: ingredients and nutritional content.
Weekly Meal Planner Component	Generates a weekly meal plans using recipes and user preferences that were used in the daily or more often.
Calorie Tracker Component	Takes care of calculating consumed calories based on the users desired recipes, instantly after the used confirms that they “ate” said meal.
Notification Component	Sends alerts such as calorie warnings and meal reminders.

Table 21: Components definitions

8.3 Responsibilities

This subsection highlights the primary responsibilities and expected behavior of each system component. It summarizes the essential duties each component must fulfill and the services it provides to the HealthyCooking system.

User Authentication Component: Is responsible for controlling all that is concerned with access control to the system. It will check the login provided by a user against written records in the system's database for validity; only authorized users should be granted access into the system. As well as registering new users by gathering necessary information, validating input, encrypting passwords, and storing user accounts in a secure manner. It provides secure logout operations to terminate active sessions. Additionally, provides 'forgot my password service', which will send verification procedures and enforces strong password policies.

User Profile Component: This is responsible for all data related to the user's personal and dietary preferences. It gathers information like: age, weight, gender, activity level, dietary preference (halal, vegetarian, allergy restrictions), and finally calorie targets. After that it will be presented in a user-friendly interface for the user to see and modify them if they want. Any modifications are first checked 'are you sure you want to save changes?' before being committed to the system database. The component ensures all other subsystems, such as the Weekly Meal Planner and Calorie Tracker, always draw upon a source of accurate, updated profile data, allowing highly customized recommendations and calculations.

Recipe Management Component: The Recipe Management Component supports the creation and entire life cycle of recipes in the system. It provides a facility for authorized users-admins or nutrition experts-to add new recipes by defining their ingredients, steps in preparation, macronutrient values, and caloric data. It allows for the editing of already saved recipes, deleting recipes that become outdated, and fetching the details of recipes for display purposes. It ensures that nutritional calculations are consistently done by automatic calculation of caloric values from ingredients and portion sizes. It filters the recipes on the basis of user dietary and allergic preferences, along with constraints imposed on them. It acts as the main repository for meal data, which is further utilized by the meal planner and calorie tracker.

Weekly Meal Planner Component: This module automatically generates a personalized, weekly meal plan for every user according to his profile information: dietary restrictions, calorie goals, and meal type preferences. Based on this input, the planner will pick recipes from the Recipe Management Component and set up daily meal schedules. It may enforce rules of optimization like calorie distribution across meals for the day, variation of ingredients used, and nutritional balance. It also supports the regeneration of the meal plan if user preferences change or if the user requests an alternative menu. Finally, the resulting weekly plan is presented clearly and can be exported or linked with other system components for tracking.

Calorie Tracker Component: This shall maintain the log of a user's daily calorie intake, record meals ingested both selected from the weekly meal plan and added manually, and calculate, using recipe data, the sum of nutritional values. Updates running totals of daily calories, macronutrients, and other nutritional metrics are reflected. It will provide real-time feedback if a user is within the daily recommended limits, exceeds them, or eats too little. Daily and weekly summaries also allow the user to get an overview of eating habits. Needless to say, this is a component closely intertwined with both the User Profile and the Weekly Meal Planner.

Notification Component: This component will be responsible for all the automated interactions between the system and the user. It will include reminders of meals on schedule, alerts of soon-to-be-reached or just overshot calorie limits, notifications of meal plans that are near expiration, and recommendations for much healthier choices. Messages will be routed through in-app alerts, email, or push notifications according to user preference. For example, this can involve scheduling notifications, formatting messages, maintaining delivery status, among others. With this notification component, a user can stay in touch with the system and get updates in due time for better usability and follow-through on the dietary goals.

8.4 Constraints

Each component has its own assumptions, limitations, and operational constraints, which are described in this subsection. These constraints may relate to data format, timing, storage requirements, or interaction behaviors within the system.

User Authentication Component

- Requires secure storage and encrypted processing of user credentials.
- Depends on stable database access for login and registration operations.
- Must enforce strict validation rules to prevent unauthorized access.

User Profile Component

- User information must meet predefined formatting and validation requirements.
- Profile updates must be committed atomically to avoid inconsistent or partially saved data.
- Sensitive data must be protected from unauthorized access.

Recipe Management Component

- Requires accurate and complete nutritional data for all recipes.
- Ingredients and measurements must follow standardized data structures.
- Cannot generate nutritional analysis without required nutritional fields.

Weekly Meal Planner Component

- Depends on the availability of sufficient recipes that meet the user's dietary needs.
- Must handle situations where dietary restrictions reduce valid meal plan options.
- Requires accurate preference data to produce valid weekly plans.

Calorie Tracker Component

- Relies on accurate nutritional values to calculate daily calorie totals.
- Requires real-time updates to ensure correct daily summaries.
- Assumes that meal entries contain complete calorie-related information.

Notification Component

- Requires operating system support for scheduling and delivering notifications.
- Alert timing depends on the reliability of the OS-level scheduler.
- May be limited by device permissions or network availability.

8.5 Composition

This subsection outlines the internal subcomponents and logical modules that comprise each major component. It describes how these internal elements interact to fulfill the responsibilities assigned to the component.

User Authentication Component: Composed of the credential validation unit, session management module, and password recovery mechanism.

User Profile Component: Composed of the profile retrieval unit, preference management module, and input validation submodule.

Recipe Management Component: Composed of the recipe creation handler, editing and deletion module, retrieval engine, and nutritional analysis unit.

Weekly Meal Planner Component: Built from the preference-matching logic, recipe filtering module, and weekly plan generation engine.

Calorie Tracker Component: Composed of the calorie computation engine, daily total updater, and summary presentation module.

Notification Component: Composed of the alert generation unit, scheduling module, and system notification interface.

8.6 Uses/Interactions

In this part, communication and coordination among key system components are illustrated using UML Sequence Diagrams. The diagrams shown below are supposed to demonstrate how the system reacts and behaves during some highlighted use cases, additionally, how components interact with each other, and how user actions flow across the system. The sequence diagrams below represent the prime or most likely used cases and interactions within the HealthyCooking application.

User Login Sequence Diagram

This sequence diagram shows the process handled by the User Authentication Component during a login attempt. It illustrates how the system verifies user credentials, retrieves the associated profile information, and manages error cases resulting from incorrect login details.

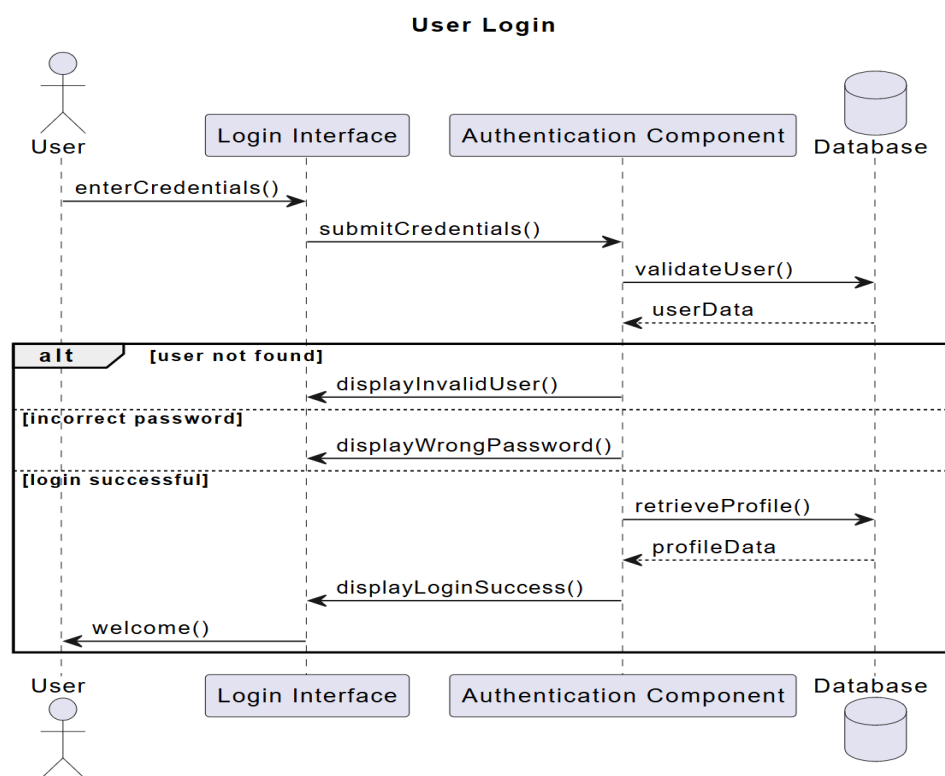


Figure 57: User Login Sequence Diagram

Update Profile and Preferences Sequence Diagram

This sequence diagram illustrates how the User Profile Component loads the user's existing information, processes updates to personal data and dietary preferences, validates the submitted input, and saves the updated profile data to the database.

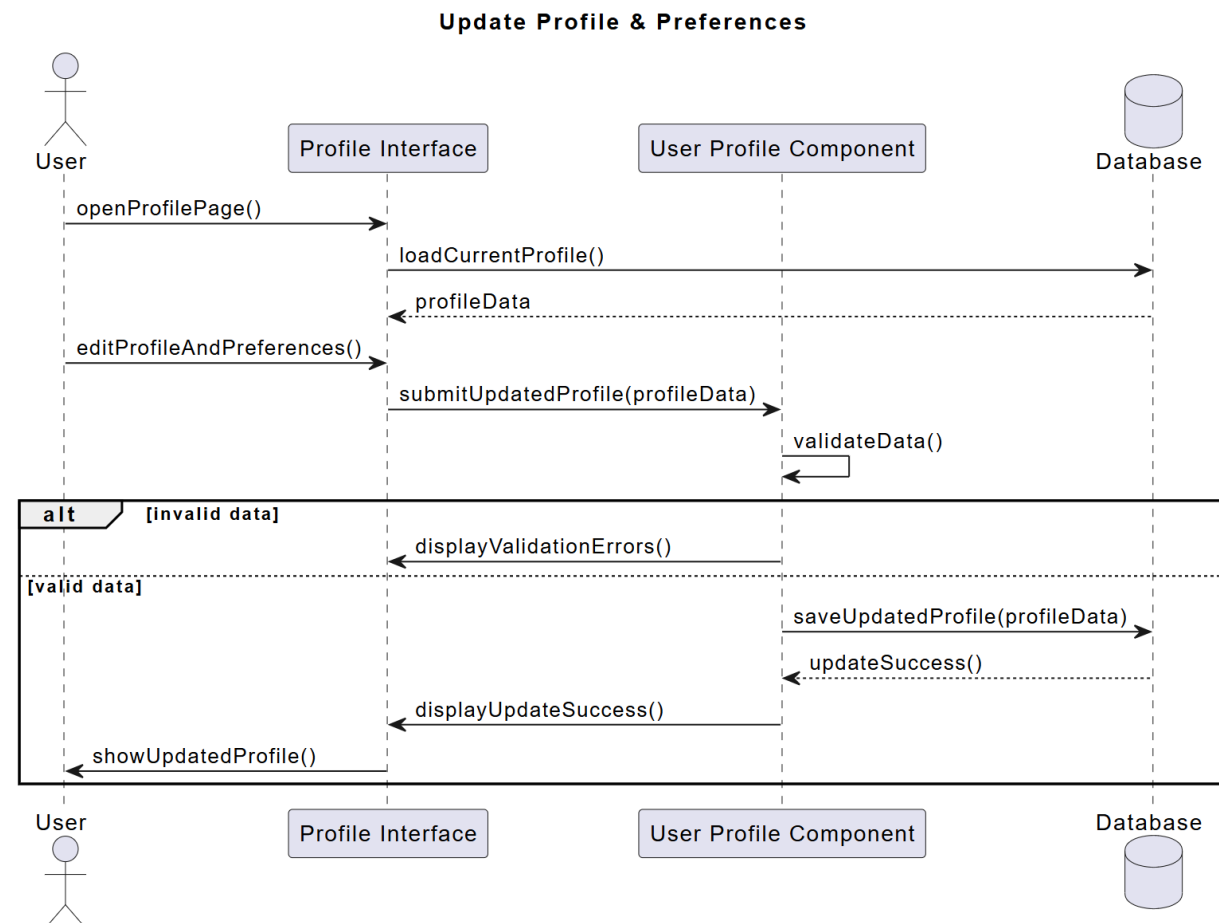


Figure 58: Update profile and preferences sequence diagram

Generate Weekly Meal Plan Sequence Diagram

This sequence diagram is explaining how the Weekly Meal Planner Component obtains the user's dietary preferences and recipes, with that in mind it will create a personalized weekly meal plan based on user stats, and finally takes care of cases in which needed recipes are unavailable, and saves the final finished plan in the system's application database.

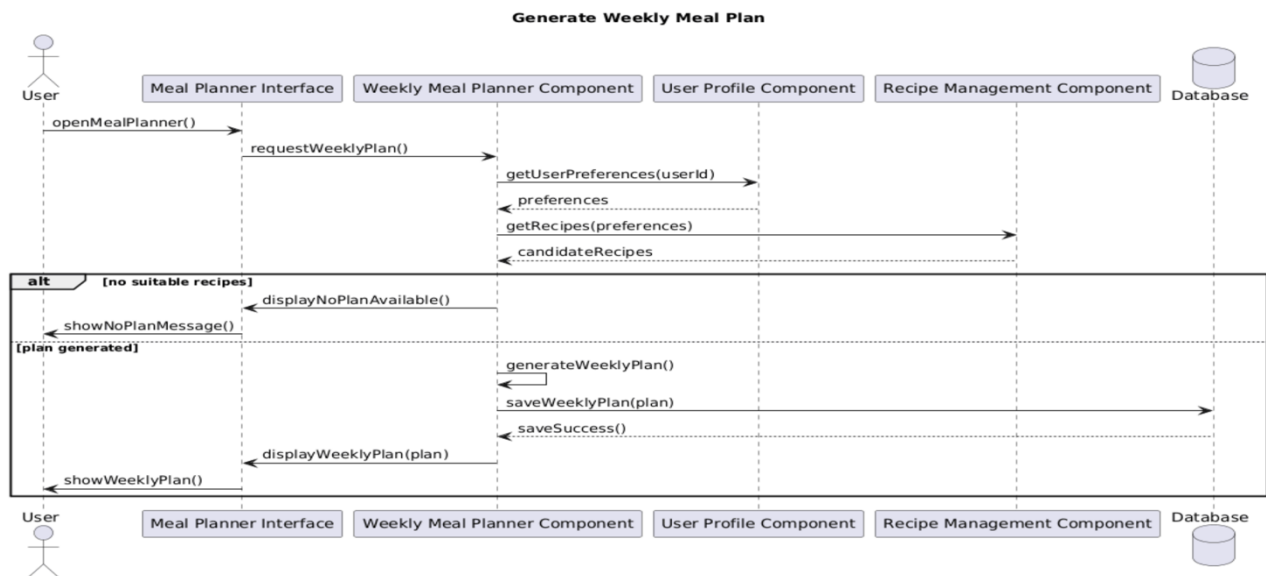


Figure 59: Generate Weekly Meal Plan Sequence Diagram

Track Calories and Send Alert Sequence Diagram

This sequence diagram is showing how the Calorie Tracker Component goes through the steps of processing the meals logged by the user, it'll update the daily calorie total if needed, and/or triggers the Notification Component to send a message to the user alerting them when the consumed calories exceed the daily limit.

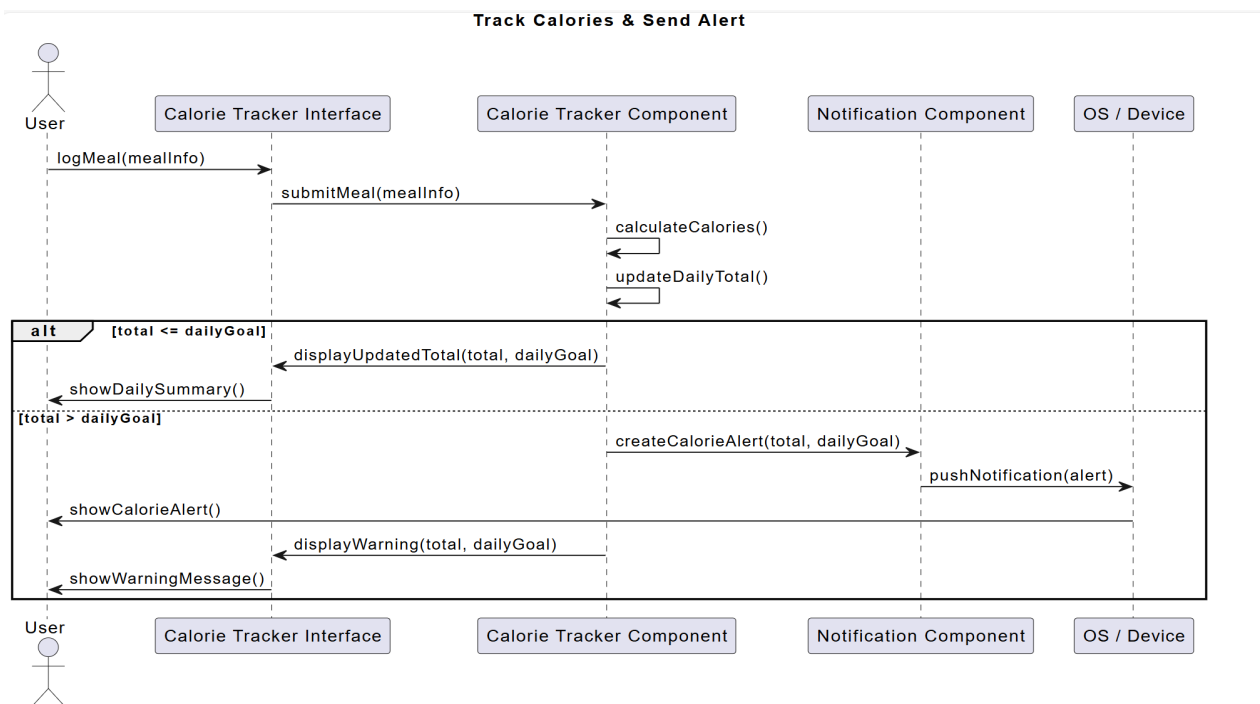


Figure 60: Track Calories and Send Alert Sequence Diagram

Browse & Search Recipes Sequence Diagram

This sequence diagram demonstrates how the system handles recipe searching. It shows how the user enters keywords, how the application queries the database for matching recipes, and how the results are returned and displayed to the user.

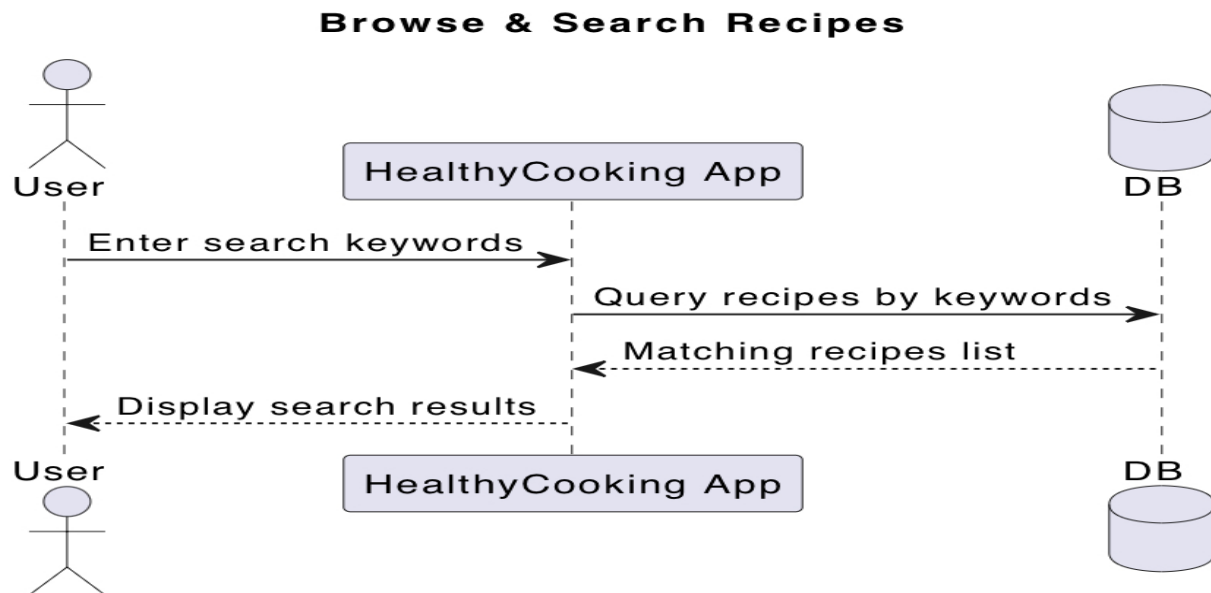


Figure 61: Browse and Search Recipes Sequence Diagram

View Recipe Details Sequence Diagram

This sequence diagram illustrates how the system retrieves and displays detailed information for a selected recipe. It shows how the user selects a recipe, how the application requests the full recipe details from the database, and how the information is shown to the user.

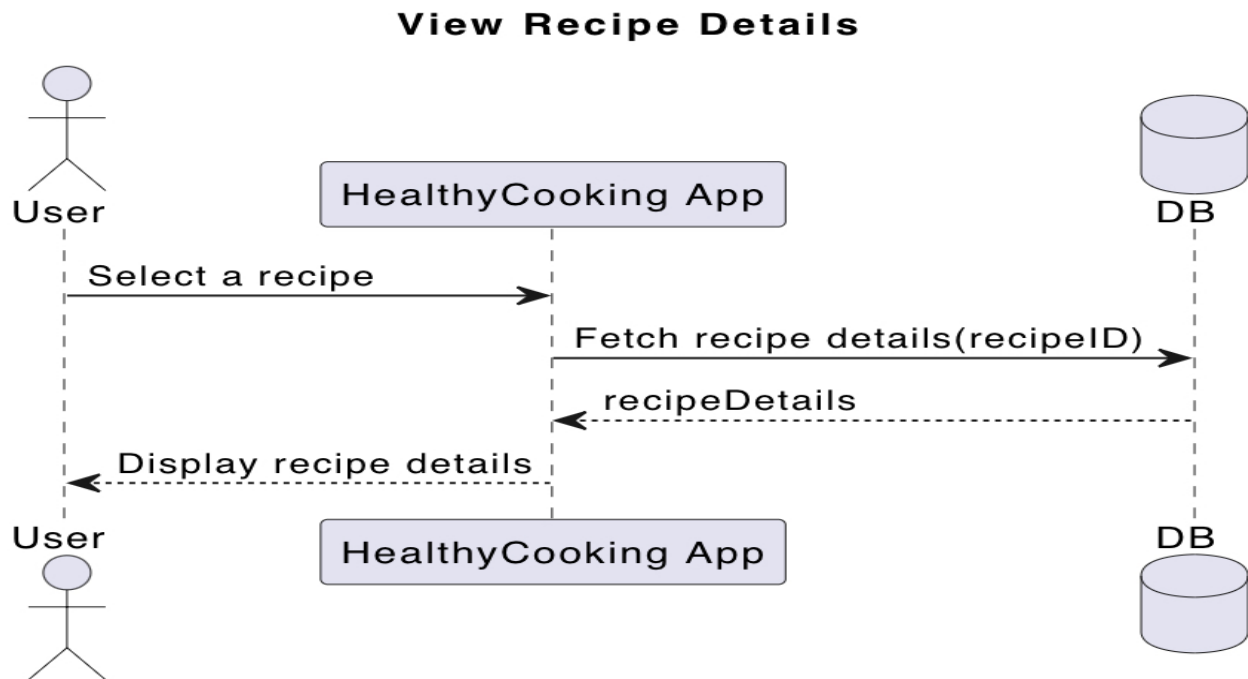


Figure 62: View Recipe Details Sequence Diagram

Add Recipe to Personal Collection Sequence Diagram

This sequence diagram explains the steps involved when a user adds a recipe to their personal saved list. It shows how the user selects a recipe to save, how the system stores it in the database, and how the application confirms that the recipe was successfully added.

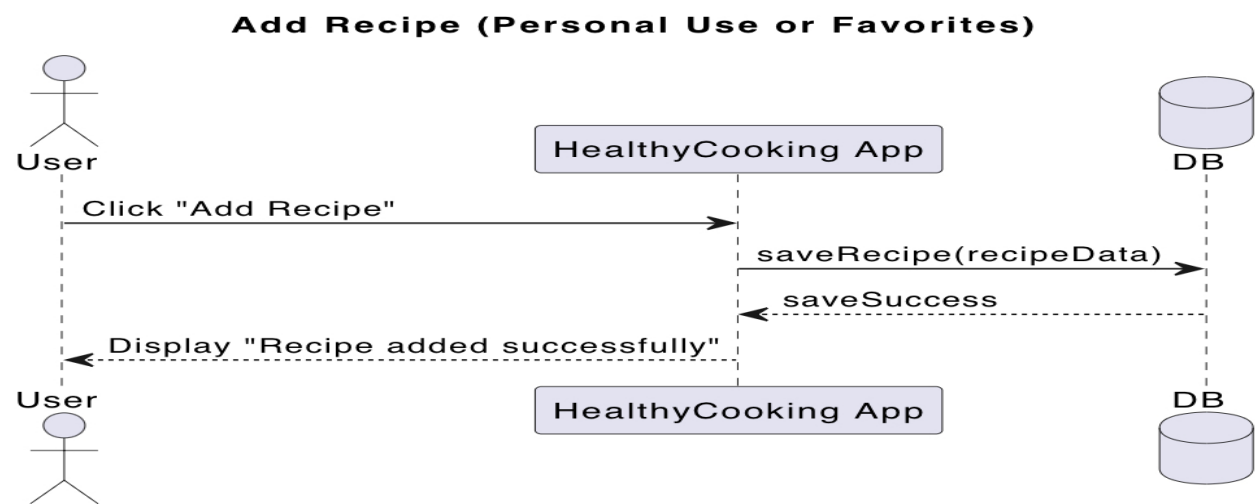


Figure 63: Add Recipe to Personal Collection Sequence Diagram

Save Recipe to Favorites Sequence Diagram

This sequence diagram demonstrates how a user marks a recipe as a favorite. It shows how the app updates the user's favorites list in the database and how the confirmation is displayed after the action is completed.

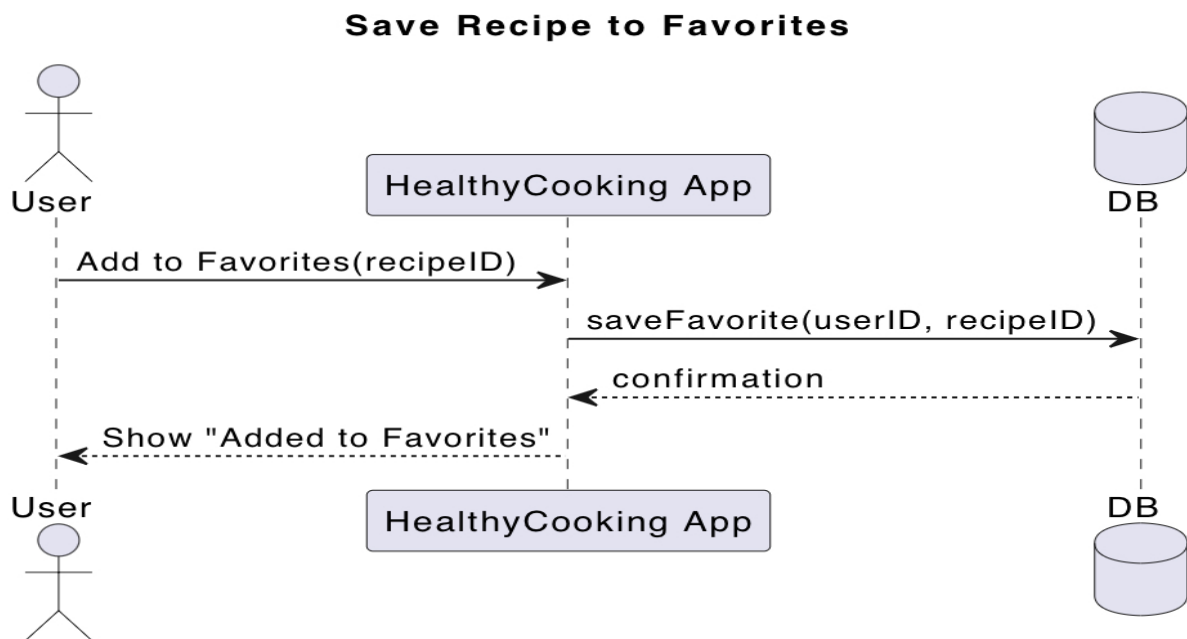


Figure 64: Save Recipe to Favorites Sequence Diagram

Remove Recipe from Favorites Sequence Diagram

This sequence diagram outlines the process of removing a recipe from the user's favorites list. It shows how the user initiates the removal, how the app updates the database, and how the system confirms that the recipe was successfully removed.

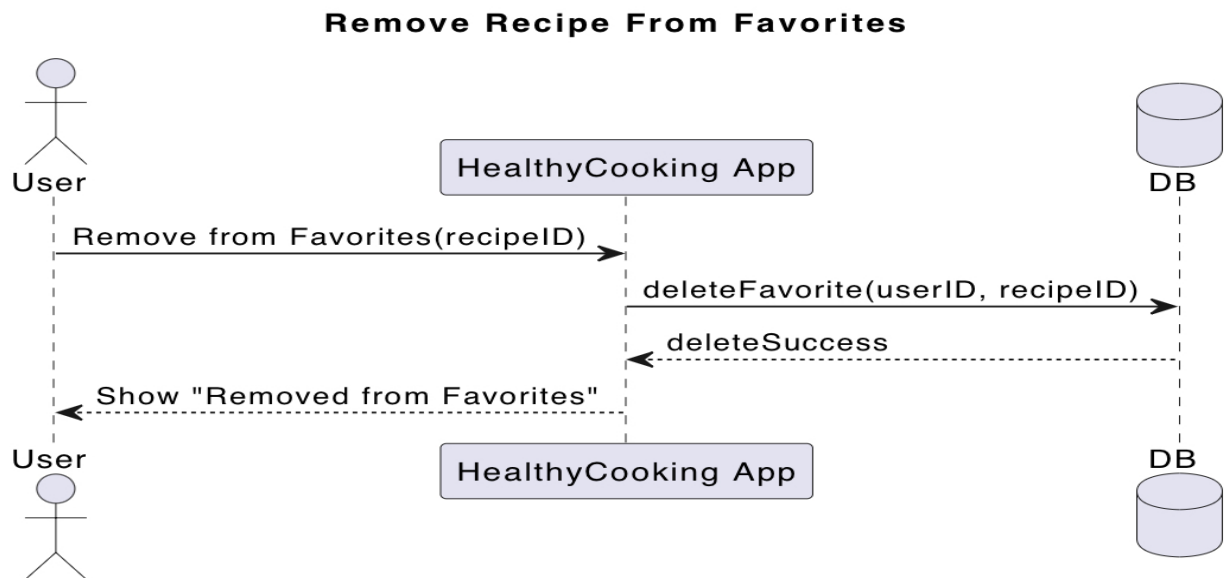


Figure 65: Remove Recipe from Favorites Sequence Diagram

Generate AI Recipe Suggestions Sequence Diagram

This sequence diagram describes how the system generates personalized recipe suggestions using AI. It shows how the app retrieves the user’s preferences, processes them using AI, requests suitable recipes from the database, and presents personalized suggestions to the user.

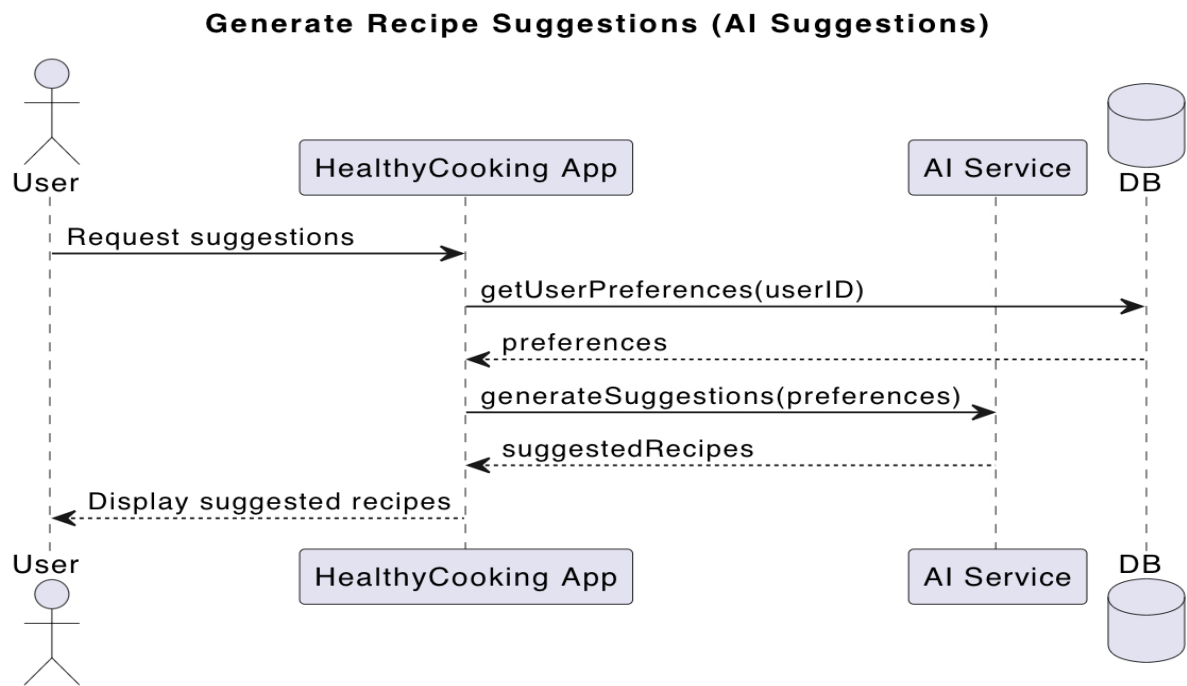


Figure 66: Generate AI Recipe Suggestions Sequence Diagram

8.7 Resources

This subsection describes the resources required by each component.

Component Name	Used By	Required Resources
User Authentication	Login/Registration System	User database, encryption libraries
User Profile	Profile Management	User profile table
Recipe Management	Meal Planner, Calorie Tracker	Recipe database, nutritional data
Weekly Meal Planner	Meal Planning System	Recipes, user dietary preferences
Calorie Tracker	Tracking System	Nutritional values, meal logs
Notification Component	Notification System	OS notification services, scheduler

Table 22: Components resources

8.8 Processing

This subsection summarizes the internal processing performed by each component. The descriptions provide a high-level overview of the logic implemented in the system, with detailed algorithms referenced in Section 7.

User Authentication Component

Performs input validation, credential verification, session generation, and password reset.

(Refer to Section 7.1: User Authentication Component)

User Profile Component

Retrieves and updates user profile information.

(Refer to Section 7.2: User Profile Component)

Recipe Management Component

Processes recipe creation, modification, deletion, and retrieval.

(Refer to Section 7.3: Recipe Management Component)

Weekly Meal Planner Component

Generates weekly plans using recipes and user preferences.

(Refer to Section 7.4: Meal Planner Component)

Calorie Tracker Component

Calculates daily calories and displays dietary summaries.

(Refer to Section 7.5: Calorie Tracker Component)

Notification Component

Schedules and sends reminders or alerts.

(Refer to Section 7.6: Notification Component)

8.9 Interface/Exports

This subsection identifies the external functions provided by each component and summarizes their purpose within the HealthyCooking system. To guarantee zero flaws operation of the system as a whole, the application developers must have a clear vision and comprehension of the features of each HealthyCooking subsystem. As a result, it is crucial to examine the restrictions (constraints), pre-conditions, post-conditions, inputs, and outputs. The system's components and their goals were thoroughly covered in the previous subsection 8.1. The restrictions, pre-conditions, and post-conditions were also discussed in section 8.2. Finally, sequence diagrams and flow charts are utilized to demonstrate the functions in subsections 8.3 and 8.7. Nevertheless, here are the functions names that are connected to the interfaces and exports, that'll be used in the system.

Function Name	Type	Purpose
signUp()	Procedural	Registers a new user
logIn()	Procedural	Authenticates user credentials
logOut()	Procedural	Logs the user out of the system
resetPassword()	Procedural	Handles password reset requests
updateProfile()	Procedural	Updates stored user data
viewProfile()	Procedural	Retrieves user profile information
addRecipe()	Procedural	Inserts a new recipe into the database
updateRecipe()	Procedural	Modifies an existing recipe
deleteRecipe()	Procedural	Removes a recipe from the database
viewRecipe()	Procedural	Retrieves recipe information
generateWeeklyPlan()	Procedural	Creates a weekly meal plan
viewWeeklyPlan()	Procedural	Retrieves weekly meal plan
calculateDailyCalories()	Procedural	Computes total calories consumed
viewDailySummary()	Procedural	Displays daily nutritional summary
sendMealReminder()	Procedural	Sends notification for scheduled meals
sendCalorieAlert()	Procedural	Sends alert when calorie goals are exceeded

Table 23: Most important system functions

8.10 Detailed Subsystem Design

The detailed subsystem design for the HealthyCooking system has already been fully presented in **Section 7 (Component Design)**. That section provides complete descriptions of all major subsystems, including:

- Internal structure
- Processing logic
- Pseudocode
- Local data tables
- Responsibilities
- Interactions with other components

Each subsystem has been described with sufficient detail to support implementation. To avoid unnecessary repetition, this section serves as a reference to the detailed design already documented. The following mapping summarizes the location of each subsystem's full internal design:

Subsystem	Detailed Design Reference
User Authentication Component	Section 7.1
User Profile Component	Section 7.2
Recipe Management Component	Section 7.3
Weekly Meal Planner Component	Section 7.4
Calorie Tracker Component	Section 7.5
Notification Component	Section 7.6

Table 24: Subsystems design reference

These sections include all required attributes such as classification, responsibilities, constraints, composition, interactions, resources, processing behavior, and interface specifications, fulfilling the Detailed Subsystem Design requirements specified by the IEEE Software Design Description standard.

9. Other Design Features

All design features of the HealtyCooking system have been discussed and illustrated in detail in the above sections of this document.

10. Requirements Traceability Matrix

A requirements traceability matrix is represented to relate the functional requirements identified in the SRS [1] to the design components and pseudocode in this SDS document. This makes sure that every feature described in the SRS is completely and accurately supported by the components of the SDS document.

Association ID in SRS	Technical Assumption / Customer Need	Functional Requirement (REQ)	User	Association ID in SDS	System Component(s)
4.6 Search Recipe	User needs to find recipes by name, ingredients, or category.	REQ-1: User enters keyword search input REQ-2: System retrieves relevant recipes	End User	7.3.5 Search Recipes	Recipe Management Component
4.7 Plan Weekly Meals	User needs help organizing weekly meals	REQ-1: User selects recipes REQ-2: User sets schedule time	End User	7.4.1 Generate Weekly Plan 7.4.2 View Weekly Plan	Meal Planner Component
4.8 Track Calories	User needs to monitor and track their daily calorie intake to maintain healthy eating habits.	REQ-1: User chooses recipe or enters meal details REQ-2: System calculates total calories REQ-3: System stores calories and updates daily summary	End User	7.5.1 Add Meal 7.5.2 calculateDailyCalories 7.5.3 viewDailySummary	Calorie Tracker Component

4.9 Profile Management	User needs to update/view their personal diet info	REQ-1: Enter profile fields REQ-2: System stores updated profile data	End User	7.2.1 Update Profile 7.2.2 View Profile	User Profile Component
4.10 Personalized Meal Recommendation	User needs meals recommended based on diet, goals, allergens	REQ-1: AI searches based on stored user data REQ-2: AI recommends meals	End User	7.4.1 Generate Weekly Plan	Meal Planner Component + AI Logic
4.11 Meal Reminders	User needs reminders for meal times	REQ-1: Choose recipe and set reminder time REQ-2: System stores reminder REQ-3: System confirms success	End User	7.4.4 Weekly Plan Notifications	Meal Planner Component

Table 25: Requirements Traceability Matrix